Logo de l'école
à déposer dans
`logos/`

ESPRIT
École Supérieure Privée d'Ingénierie et de Technologie

RAPPORT DE STAGE DE FIN DE CYCLE

Service de données climatiques par bâtiment

Concevoir et industrialiser une jointure aléa × vulnérabilité à l'échelle du bâtiment pour les assureurs français

De la maquette de démonstration à un contrat de données traçable, testé et déployé sur une infrastructure serverless.

Présenté par	**Hazem Mrad**
Intitulé du diplôme	Diplôme d'ingénieur
Cycle	Cycle d'ingénieur
Structure d'accueil	Groupe Talan — Conseil et ingénierie technologique
Période de stage	14 août 2026 – 15 septembre 2026
Tuteur pédagogique	.....
Tuteur entreprise	.....
Année universitaire	**2026–2027**

Remerciements

Ce stage d'été n'aurait pas pris cette forme sans plusieurs personnes que je souhaite remercier ici.

Je remercie **Talan** de m'avoir accueilli dans le cadre du SummerCamp 2026 et de m'avoir laissé une latitude réelle sur le sujet : proposer une problématique, la reformuler, puis la construire jusqu'au déploiement. Un stage où l'on peut supprimer du code parce qu'il ne sert pas le produit vaut plus qu'un stage où l'on empile des fonctionnalités.

Je remercie les personnes de l'écosystème assurance — l'actuaire et le professionnel de l'assurance qui ont accepté de réagir au projet au démarrage. Leurs réponses, parfois dérangeantes, ont directement réécrit la direction produit : le « score » que je pensais être le livrable a été explicitement refusé, et cette conversation a probablement été le meilleur investissement des deux mois.

Je remercie enfin les équipes et les projets open data qui rendent ce travail possible : Géorisques, la BDNB du CSTB, la Géoplateforme de l'IGN, Sandre, Panoramax, Open-Meteo et le Copernicus Climate Data Store. Un service comme Typhoon ne remplace aucune de ces sources : il existe parce qu'elles sont publiques, documentées et réutilisables.

Hazem Mrad

Résumé

Ce rapport présente le travail réalisé pendant un stage d'été au sein du programme SummerCamp 2026 de Talan. Le sujet est un service de données climatiques **par bâtiment** destiné aux assureurs français, baptisé Typhoon.

Le problème de départ est concret. Les données publiques existent et sont gratuites : Géorisques publie les aléas réglementaires, la BDNB décrit les bâtiments, Copernicus produit des projections climatiques. Mais aucune de ces sources ne relie l'exposition d'un bâtiment à sa vulnérabilité. Géorisques ne sait pas ce qu'est le bâtiment ; la BDNB ne sait pas s'il est dans un périmètre d'inondation. Aujourd'hui, un souscripteur résout ce manque en ouvrant deux onglets et en jugeant à l'œil.

La contribution du stage tient en trois apports. D'abord, une **reformulation produit** : le livrable n'est pas une interface, c'est un contrat de données unique, l'adresse en entrée, un JSON canonique en sortie, où chaque fait porte sa source et son niveau de résolution, et où aucun score composite n'apparaît — la notation reste le métier de l'assureur. Ensuite, une **industrialisation de la jointure** : un connecteur WFS Géorisques (point-in-polygon écrit à la main, parseur GML 3.2) croisé avec un passage verbatim de la BDNB, 13 aléas normalisés, un cache, un quota, une authentification par clé et une suite de tests hors-ligne. Enfin, une **couche de mise en situation** sur la carte : des classes de retour officielles du TRI, une timeline horaire, un moteur où le scénario pilote le niveau et la pluie n'en pilote que la forme, et un mode de rendu cinématique explicitement étiqueté comme non contractuel.

Le résultat est une application déployée et vérifiée en ligne, un backend de 70 fichiers Python, un frontend de 68 modules TypeScript, **214 tests backend et 143 tests frontend** hors-ligne, une constitution d'ingénierie qui interdit les scores dans le contrat sérialisé, et un ensemble de décisions documentées — y compris celles qui ont consisté à *supprimer* du code livré.

Mots-clés : assurance, risque climatique, open data, Géorisques, BDNB, TRI, jointure spatiale, FastAPI, React, Mapbox GL JS, serverless.

Synthèse en une page

Typhoon — données climatiques par bâtiment

Une adresse, treize aléas, un bâtiment, une provenance sur chaque fait.

Le problème

Géorisques publie les périmètres d'aléas mais ne connaît pas le bâtiment. La BDNB décrit les bâtiments mais ne sait pas lesquels sont inondables. Aucune source ne relie exposition et vulnérabilité à l'échelle du bâtiment. Un souscripteur comble ce vide à l'œil, sans traçabilité.

La réponse

Un service qui prend une adresse française et renvoie un contrat JSON canonique : treize aléas normalisés, le passage verbatim de la BDNB, une provenance et un niveau de résolution sur chaque fait — et aucun score.

Les trois décisions structurantes

1. **Refuser le score composite.** Sous Solvabilité II, un score tiers engage son producteur et se fait reconstruire ou refuser par l'actuaire. Le champ de résolution devient le signal de qualité.
2. **Faire de l'incertitude une fonctionnalité.** Trois états (présent / absent / inconnu) et une distinction stricte entre absence de risque et panne de service.
3. **Soustraire plutôt qu'ajouter.** La phase la plus productive du stage a consisté à supprimer quatre connecteurs, une couche de notation et un tableau de bord.

Chiffres clés

357	tests automatisés hors-ligne	**28**	exigences fonctionnelles
214	tests backend (pytest)	**18**	tâches planifiées (+10 scénarios)
143	tests frontend (Vitest)	**13**	aléas normalisés
70	fichiers Python (backend)	**4**	aléas certifiés
68	modules TypeScript (frontend)	**9**	sources externes intégrées

Résultat

Un service déployé et vérifié en ligne le 15 septembre 2026, couvrant l'ensemble du parcours : géocodage, jointure géométrique, fiche de vulnérabilité, mise en situation cartographique et export. Le prototype démontre une thèse : la donnée publique française permet de produire une information de risque climatique par bâtiment, traçable et défendable — à condition d'accepter de ne pas tout savoir et de le dire.

Ce que le prototype n'est pas : un produit commercialisable. L'authentification est simulée, le magasin de lots est en mémoire, la certification attend sa qualification humaine et le budget de latence n'est pas calibré. Ces limites sont détaillées au chapitre 9 et assumées.

Table des matières

Remerciements **1**

Résumé **2**

Synthèse en une page **3**

1 Contexte du stage **12**
1.1 La structure d'accueil . . . 12
1.2 Le problème posé . . . 12
1.2.1 Trois sources, aucune jointure . . . 12
1.2.2 Le contournement actuel . . . 13
1.3 La mission . . . 13
1.4 Démarche et organisation du rapport . . . 13

2 Le métier de l'assurance et les invariants du produit **15**
2.1 Deux acheteurs, un moteur . . . 15
2.1.1 Le souscripteur (*underwriter*) . . . 15
2.1.2 L'actuaire (*pricing*) . . . 15
2.2 La non-stationnarité : pourquoi la donnée historique ne suffit pas . . . 15
2.3 Les invariants du produit . . . 16
2.3.1 Faits et provenance uniquement — pas de score . . . 16
2.3.2 Un niveau de résolution sur chaque fait . . . 16
2.3.3 Passage verbatim de la BDNB . . . 17
2.3.4 Une seule forme canonique . . . 17
2.3.5 Le lot est N fois l'unitaire . . . 17
2.3.6 Honnêteté sur le RGA . . . 18
2.3.7 Pas de code dormant . . . 18
2.4 Synthèse . . . 18

3 Analyse de l'existant et recentrage **19**
3.1 État de départ : un prototype fonctionnel . . . 19
3.2 Le diagnostic : quatre symptômes, une cause . . . 19
3.2.1 La redondance avec les portails sources . . . 19
3.2.2 Le score composite . . . 19
3.2.3 Le tableau de bord climatique . . . 19
3.2.4 La décoration 3D . . . 20
3.2.5 La cause commune . . . 20
3.3 La décision de recentrage . . . 20
3.4 L'exécution : supprimer . . . 20
3.5 Le compromis assumé : ce qui n'a pas été supprimé . . . 21
3.6 Enseignement . . . 21

4 Méthodologie d'ingénierie **22**
4.1 Le problème que la méthode résout . . . 22
4.2 Le triptyque documentaire . . . 22

4.2.1 La constitution . . . 22
4.2.2 La spécification . . . 23
4.2.3 Le plan . . . 23
4.3 Le flux de travail . . . 23
4.4 Convention de commit . . . 24
4.5 Décisions documentées . . . 24
4.6 Outils . . . 24
4.7 Apport de la méthode . . . 24

5 Les sources de données et leur intégration 26
5.1 Vue d'ensemble . . . 26
5.2 Géorisques : le cœur de la jointure . . . 26
5.2.1 Ce que la source fournit . . . 26
5.2.2 Le test point-in-polygon . . . 26
5.2.3 Les treize aléas . . . 27
5.2.4 La ventilation par type de PPR . . . 27
5.2.5 Le piège de la boîte englobante . . . 27
5.2.6 Le piège du client HTTP partagé . . . 28
5.2.7 Une réponse vide est ambiguë . . . 28
5.2.8 Les arrondissements . . . 28
5.3 La BDNB : la vulnérabilité . . . 28
5.4 Les classes TRI : la profondeur d'eau réglementaire . . . 29
5.5 Les sources complémentaires . . . 29
5.6 Contraintes juridiques et quotas . . . 29
5.6.1 Licence Ouverte 2.0 . . . 29
5.6.2 Quotas . . . 30
5.7 Le cache . . . 30

6 Architecture et contrat de données 31
6.1 Le principe : une seule primitive . . . 31
6.2 Vue en couches . . . 31
6.3 Structure du dépôt . . . 31
6.4 Le contrat canonique . . . 32
6.4.1 Une adresse, treize aléas, un bâtiment . . . 32
6.4.2 La sémantique du champ `present` . . . 33
6.4.3 La règle de sérialisation des champs optionnels . . . 33
6.4.4 Le schéma comme garde-fou . . . 34
6.5 Le pipeline unitaire . . . 34
6.5.1 Le géocodage d'abord, et comme point de non-retour . . . 34
6.5.2 Collecte parallèle . . . 34
6.5.3 Normalisation . . . 34
6.6 Les routes de l'API . . . 35
6.7 Sécurité . . . 35
6.7.1 Authentification par clé . . . 35
6.7.2 Défenses en profondeur . . . 35
6.8 Le mode dégradé . . . 35

7 Le moteur de scénarios interactifs 37
7.1 Objectif et référence . . . 37
7.2 Le bus d'état unique . . . 37
7.3 Bug 1 : les clés de scénario ne correspondaient pas . . . 38
7.3.1 Symptôme . . . 38

7.3.2 Cause . . . 38
7.3.3 Correction . . . 38
7.4 Bug 2 : le seuil de dommages masquait la carte . . . 38
7.4.1 Symptôme . . . 38
7.4.2 Cause . . . 38
7.4.3 Correction . . . 38
7.5 Bug 3 : « rien ne se passe » par temps sec . . . 39
7.5.1 Symptôme . . . 39
7.5.2 Cause — deux causes distinctes . . . 39
7.5.3 Corrections . . . 39
7.6 Le mode VFX et l'isolation des modes . . . 39
7.7 L'absence n'est pas une panne . . . 40
7.8 Les autres aléas . . . 40
7.8.1 L'ouragan . . . 40
7.8.2 Le feu . . . 40
7.9 Synthèse des décisions du moteur . . . 41

8 Le mode de rendu cinématique (VFX) 42
8.1 Le problème : la tension . . . 42
8.2 La solution : gouverner la contradiction . . . 42
8.2.1 Deux modes obligatoires . . . 42
8.2.2 Règles d'entrée . . . 43
8.2.3 Le bus d'entrée unique . . . 43
8.3 Contraintes d'intégration . . . 43
8.3.1 Exclusion mutuelle avec la couche météo . . . 43
8.3.2 Le montage et le démontage . . . 44
8.4 Étiquetage . . . 44
8.5 État d'avancement . . . 44
8.6 Bilan de la démarche . . . 45

9 Qualité, tests et limites connues 46
9.1 La stratégie de test . . . 46
9.1.1 Hors-ligne par défaut . . . 46
9.1.2 La dégradation testée aussi rigoureusement que le nominal . . . 46
9.1.3 Les tests d'abord . . . 46
9.2 Inventaire de la suite . . . 47
9.3 La suite de certification . . . 47
9.4 Les portes de non-régression . . . 48
9.4.1 Porte de recherche sur les champs interdits . . . 48
9.4.2 Porte de recherche sur les surfaces supprimées . . . 48
9.5 Le test d'équivalence du lot . . . 48
9.6 Les limites connues . . . 48
9.7 Performance . . . 49

10 Déploiement et mise en production 51
10.1 L'objectif : une URL, zéro configuration . . . 51
10.2 Le principe du mono-domaine . . . 51
10.3 Les obstacles rencontrés . . . 52
10.4 Comportements propres au serverless . . . 52
10.5 Vérification de l'instance en ligne . . . 53
10.6 Adresses de référence . . . 53
10.7 Alternative de déploiement . . . 53

11 Résultats et validation **54**
11.1 Le livrable . . . 54
11.2 Le contrat en pratique . . . 54
11.3 Ce que le produit démontre . . . 55
11.3.1 La jointure existe et elle est vérifiée . . . 55
11.3.2 Le refus du score est tenable . . . 55
11.3.3 Les modes de défaillance sont explicites . . . 55
11.3.4 Le moteur de scénarios relie la donnée au jugement . . . 55
11.4 Chiffres clés . . . 56
11.5 Validation . . . 56
11.6 Apports personnels . . . 56
11.7 Impact potentiel . . . 57

12 Bilan critique **58**
12.1 Ce qui a bien fonctionné . . . 58
12.1.1 La remise en question du cadrage . . . 58
12.1.2 La confrontation au terrain métier . . . 58
12.1.3 La discipline documentaire . . . 58
12.2 Ce qui a mal fonctionné . . . 58
12.2.1 Les bugs sémantiques répétés . . . 58
12.2.2 La dérive initiale . . . 59
12.2.3 L'instabilité du déploiement . . . 59
12.2.4 Le périmètre du mode VFX . . . 59
12.3 Ce que je ferais différemment . . . 60
12.4 Compétences acquises . . . 60
12.5 Appréciation personnelle . . . 60

13 Conclusion et perspectives **62**
13.1 Rappel de la problématique . . . 62
13.2 Réponse apportée . . . 62
13.3 Les trois idées à retenir . . . 62
13.3.1 Le refus du score est une position tenable . . . 62
13.3.2 L'honnêteté sur l'incertitude est une fonctionnalité . . . 63
13.3.3 La soustraction est un travail d'ingénierie . . . 63
13.4 Perspectives . . . 63
13.4.1 Court terme — finir et qualifier . . . 63
13.4.2 Moyen terme — rendre le service exploitable . . . 63
13.4.3 Long terme — le produit complet . . . 63
13.5 Mot de la fin . . . 64

A Référence du contrat canonique **65**
A.1 Structure complète . . . 65
A.2 Champs interdits . . . 66
A.3 Règles de sérialisation . . . 66
A.4 Le référentiel des treize aléas . . . 67

B Surface de l'API et mise en œuvre **68**
B.1 Points d'entrée . . . 68
B.2 Format des erreurs . . . 68
B.3 Commandes d'installation . . . 68
B.4 Commandes de vérification . . . 69
B.5 Vérification de l'instance déployée . . . 69

B.6 Reproduction des captures d'écran 69

C Glossaire **70**
C.1 Termes métier 70
C.2 Termes techniques 70
C.3 Sigles 72

Sources et références **73**

Table des figures

1.1 Les trois phases du stage. La phase 2 est une phase de soustraction : elle retire davantage qu'elle n'ajoute, et c'est ce qui a rendu le produit défendable. 14

3.1 Le recentrage en une image. Tout ce qui ne sert pas la jointure aléa × vulnérabilité est retiré du périmètre ou relégué à la couche de présentation. 21

6.1 Architecture en quatre couches. La règle d'or : aucune couche supérieure n'appelle une source externe directement, et aucun fait ne remonte sans sa provenance. . . . 31

6.2 Le pipeline unitaire. Les sorties dégradées (en ambre) traversent le même chemin que les sorties nominales : il n'existe pas de code séparé pour les cas d'échec. . . . 34

7.1 Étape 2 — simulation de scénario. À gauche, les classes TRI officielles ; à droite, la timeline horaire et le bouton de lecture. L'eau monte sur le bâtiment par extrusion Mapbox. 37

7.2 Un seul bus d'état, un seul moteur. La carte et les panneaux ne peuvent pas diverger. 38

10.1 Architecture de déploiement mono-domaine. Tout passe par une seule URL : pas de CORS, pas de variable d'environnement côté client. 51

11.1 Étape 1 — diagnostic. Les 13 aléas Géorisques avec leur badge de résolution, la fiche BDNB du bâtiment, et une photographie terrain Panoramax orientée sur le point, sur une carte 3D. 54

Liste des tableaux

2.1 Les trois niveaux de résolution et leur signification exacte. 17
2.2 Décisions métier verrouillées lors de la phase de cadrage, et leur traduction en règle technique. 18

3.1 Suppressions de la phase de recentrage, par catégorie. 20

4.1 Étiquettes d'application des règles constitutionnelles. 22
4.2 Exemples de décisions documentées et leur motivation. 24
4.3 Outils et pratiques du projet. 24

5.1 Les sources de données intégrées, leur rôle et leur niveau de résolution. 26
5.2 Les classes TRI officielles et leur signification. 29
5.3 Quotas amont et mécanismes de protection implémentés. 30

6.1 Les trois états de `present` et leur signification. 33
6.2 Surface de l'API backend. 35
6.3 Matrice des erreurs : ce que le client observe dans chaque situation. 36

7.1 Les messages distincts pour l'absence d'exposition. 40
7.2 Décisions verrouillées du moteur de scénarios. 41

8.1 Les deux modes de rendu et leurs contrats respectifs. 42
8.2 État d'avancement des tâches du mode VFX. 45

9.1 Les modules de test backend et leur objet. 47
9.2 Volumétrie de la suite de tests au terme du stage. 47
9.3 Limites connues du système au terme du stage. 49

10.1 Obstacles de déploiement et résolutions. 52
10.2 Contrôles de l'instance déployée, vérifiés le 15 septembre 2026. 53
10.3 Adresses de référence pour la démonstration. 53

11.1 Lecture du contrat pour l'adresse de démonstration principale. 55
11.2 Synthèse quantitative du projet. 56

12.1 Choix passés et alternatives envisagées. 60
12.2 Compétences mobilisées et développées. 60

A.1 Règles de sérialisation à respecter par tout consommateur. 66
A.2 Les treize aléas, leur libellé et leur niveau de résolution attendu. 67

B.1 Points d'entrée de l'API, avec authentification requise. 68
B.2 Codes d'erreur et signification. 68

C.1 Sigles et acronymes utilisés dans le rapport. 72

CHAPITRE 1

Contexte du stage

1.1 La structure d'accueil

Talan est un groupe de conseil et d'ingénierie technologique dont l'activité couvre le conseil en organisation, l'intégration de systèmes et, de plus en plus, la *data* et l'intelligence artificielle. Ses interventions s'adressent à plusieurs secteurs, dont la banque, l'assurance, l'énergie et les transports.

Ce stage s'est déroulé dans le cadre du **SummerCamp 2026**, le programme d'été du groupe. Le format est celui d'un stage projet : un sujet ouvert, une durée courte — environ un mois de travail effectif, du 14 août au 15 septembre 2026 d'après l'historique du dépôt — et une obligation de résultat tangible. Contrairement à un stage d'observation, il fallait livrer quelque chose qui tourne : un service déployé, pas une présentation.

Le sujet proposé portait sur le **risque climatique appliqué à l'assurance**, un thème qui n'est pas un exercice académique. La France a effectivement ouvert une discussion de politique publique sur l'adaptation du régime d'indemnisation des catastrophes naturelles (CatNat) au changement climatique. Travailler sur la donnée climatique appliquée au bâti, dans ce contexte, n'est pas un problème inventé pour un stage : c'est un problème que des professionnels cherchent activement à outiller.

1.2 Le problème posé

1.2.1 Trois sources, aucune jointure

Le point de départ du projet est un constat simple, mais qui n'avait pas été exploité. La France dispose d'un écosystème open data remarquable sur le risque :

— **Géorisques** publie les aléas réglementaires : périmètres de plans de prévention des risques (PPR), sites et sols pollués, réseaux de transport de matières dangereuses, cavités, retrait-gonflement des argiles, etc. La donnée est servie en WFS, donc interrogeable géométriquement.

— La **BDNB** (Base de Données Nationale des Bâtiments, CSTB) décrit chaque bâtiment : usage, hauteur, emprise, matériaux de construction, année, performance énergétique, avec des indicateurs de fiabilité.

— **Copernicus** et **Open-Meteo** fournissent la dimension temporelle : projections climatiques à 2050 et 2100, prévisions météorologiques horaires, débits de rivière.

Aucune de ces sources ne fait la jointure. Géorisques ne sait pas ce qu'est le bâtiment : il dit qu'un périmètre existe, pas si *ce* bâtiment est dedans. La BDNB ne sait pas si le bâtiment est inondable. Copernicus travaille sur une grille d'environ 10 km, donc ne dit rien d'un bâtiment précis.

La jointure aléa × vulnérabilité est le produit. Un aléa sans vulnérabilité est une carte. Une vulnérabilité sans aléa est une fiche technique. Les deux ensemble, pour une même adresse, avec la provenance de chaque fait, constituent une information qui n'existe nulle part ailleurs.

1.2.2 Le contournement actuel

Un souscripteur qui reçoit un dossier d'assurance habitation ou professionnelle doit aujourd'hui évaluer l'exposition du bien. La pratique observée — décrite dans les notes de cadrage du projet et confirmée lors des échanges avec un professionnel du secteur — consiste à ouvrir Géorisques dans un onglet, la BDNB dans un autre, et à juger visuellement : la rivière est-elle proche ? le périmètre enveloppe-t-il la parcelle ? le bâtiment est-il en rez-de-chaussée ?

C'est lent, non reproductible, non traçable, et cela ne passe pas l'épreuve d'un audit. Sous Solvabilité II, une décision de souscription doit pouvoir être justifiée. « J'ai regardé la carte et ça m'a paru bon » n'est pas une justification.

1.3 La mission

La mission confiée peut se formuler ainsi :

> *Concevoir un service qui, à partir d'une adresse française, produit une fiche de risque climatique* ***au niveau du bâtiment****, en croisant les aléas réglementaires et la vulnérabilité du bâti, avec une provenance explicite pour chaque fait — et le rendre utilisable par un souscripteur dans son travail quotidien.*

Cette formulation est volontairement large. Elle laisse ouvertes trois questions qui se sont révélées être les vraies décisions du stage :

1. **Quel est le livrable ?** Une interface graphique, un rapport narratif, ou une donnée structurée ? La réponse n'était pas évidente au départ.
2. **Quel est le niveau de résolution acceptable ?** Faire du per-bâtiment partout est-il possible ? Sinon, comment le dire honnêtement ?
3. **Que fait-on de la synthèse ?** Un assureur attend souvent un « score ». Est-ce le bon livrable ?

1.4 Démarche et organisation du rapport

Le stage s'est déroulé en trois mouvements, qui structurent ce rapport.

Phase 1 — Exploration (mi-août 2026).
Construction d'une première version fonctionnelle : quatre connecteurs de données, une couche de notation, un tableau de bord climatique, des extrusions 3D. Le produit *marchait*, mais son intérêt était difficile à défendre : il reproduisait, à résolution égale et gratuitement, ce que chaque portail source fournissait déjà.

Phase 2 — Recentrage (25 août 2026).
Une remise à plat complète. Le livrable a été redéfini comme un contrat de données, le score composite a été supprimé, et une grande partie du code de la phase 1 a été *supprimée*. Une constitution d'ingénierie a été écrite pour verrouiller ces choix par des tests et des règles non négociables.

Phase 3 — Industrialisation et mise en scène (fin août à mi-septembre). Durcissement du contrat, ajout d'un moteur de scénarios interactifs sur la carte, d'un mode de rendu cinématique, et d'un déploiement serverless vérifié en ligne.

Le rapport suit cette logique. Le chapitre 2 décrit le métier de l'assurance et ce que le produit doit — ou ne doit pas — être. Le chapitre 3 analyse l'existant et la décision de recentrage. Les chapitres 4 à 6 traitent de la méthode, des sources de données et de l'architecture. Les chapitres 7 et 8 décrivent le moteur de scénarios et le mode VFX. Le chapitre 9 détaille la stratégie de test. Le chapitre 10 couvre le déploiement, le chapitre 11 les résultats mesurés, et le chapitre 12 le bilan critique.

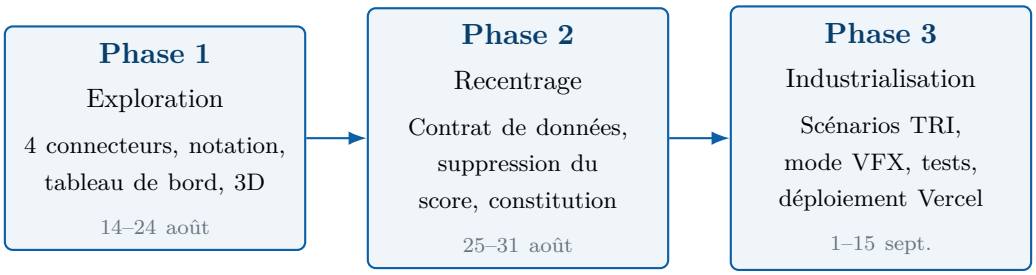


Figure 1.1. Les trois phases du stage. La phase 2 est une phase de soustraction : elle retire davantage qu'elle n'ajoute, et c'est ce qui a rendu le produit défendable.

CHAPITRE 2

Le métier de l'assurance et les invariants du produit

Un projet technique appliqué à un métier se juge d'abord sur sa compréhension de ce métier. Ce chapitre expose ce que j'ai appris du secteur, comment cela a déterminé les choix de conception, et les règles qui en découlent — règles que le projet s'interdit de violer.

2.1 Deux acheteurs, un moteur

Les échanges menés au démarrage du stage ont fait apparaître que « l'assureur » n'est pas un utilisateur unique. Deux profils aux besoins différents cohabitent.

2.1.1 Le souscripteur (*underwriter*)

C'est l'utilisateur principal de la version 1. Il traite des dossiers : une adresse arrive, il doit décider *accepter*, *mettre à l'étude* ou *refuser*. Ses contraintes :

— **rapidité** — la décision s'insère dans un flux de travail, pas dans une analyse hebdomadaire ;
— **jugement spatial** — il pense en distance, en périmètre, en relief. Il regarde une carte, il ne lit pas un tableau de bord ;
— **traçabilité** — sa décision peut être auditée. Il doit pouvoir dire d'où vient chaque élément.

C'est pour ce profil que la carte reste une fonctionnalité de premier plan, et non une décoration : elle ne montre pas les couches sources, elle montre *la jointure* — l'empreinte du bâtiment colorée par le résultat du test géométrique, accompagnée d'une fiche de vulnérabilité.

2.1.2 L'actuaire (*pricing*)

Profil secondaire, hors périmètre de la version 1 mais structurant. Il construit des modèles de tarification et de provisionnement. Ses besoins sont presque opposés :

— **pas d'interface** — il veut de la donnée, pas un écran ;
— **volume** — il traite des portefeuilles entiers, pas une adresse ;
— **variables brutes** — il veut appliquer *ses* formules.

Le point décisif : l'actuaire interrogé ne voulait *pas* de notre score. Il voulait les variables par aléa, par horizon et par scénario climatique, étiquetées et propres, pour exécuter ses propres calculs. Cette réponse a directement conduit à l'invariant central du produit.

2.2 La non-stationnarité : pourquoi la donnée historique ne suffit pas

Le raisonnement qui fonde l'intérêt du produit mérite d'être explicité, car il n'est pas immédiat.

Un assureur possède un actif que nous n'aurons jamais : l'historique de ses sinistres. Pour un bien déjà couvert, cet historique est un meilleur prédicteur que n'importe quelle reconstitution à partir de sources publiques. Il serait donc absurde de prétendre « mieux connaître le risque actuel ».

Mais le risque climatique est **non stationnaire** : le futur ne ressemble pas au passé. Un historique de sinistres, aussi volumineux soit-il, décrit des pertes réalisées sous le climat d'hier. Il ne peut structurellement pas indiquer comment l'exposition d'un bâtiment donné évolue sur les 25 à 75 prochaines années. Cette information doit venir de la science du climat et des modèles de projection, pas d'un registre de sinistres.

L'argument de vente n'est donc pas « plus de données ». C'est « la partie de l'information qui ne peut pas venir de votre historique ». Aucune quantité de données propriétaires ne comble ce trou précis.

C'est pour cette raison que la trajectoire temporelle — aujourd'hui, 2050, 2100 — est le cœur du produit, et non un ornement. Et c'est pour cette raison que la donnée climatique, un temps écartée du périmètre, a dû y revenir sous une forme honnête (voir chapitre 5).

2.3 Les invariants du produit

Le projet a produit un document de référence, la *constitution* (`constitution.md`), qui fixe les règles non négociables. Chaque règle porte un mécanisme d'application : un test automatisé, une porte CI, ou une revue humaine. En voici les principales, avec leur justification.

2.3.1 Faits et provenance uniquement — pas de score

Invariant produit

Aucune réponse de l'API ne peut contenir de score composite, de jugement de risque, de recommandation, ni d'évaluation générée par IA. La graduation fournie par la source est transmise verbatim sous forme de texte ; toute conversion en bandes est de la coloration de présentation et n'est *jamais* sérialisée dans le contrat canonique.

La justification est double. Juridiquement, un score tiers de risque climatique, produit par un acteur qui n'est ni l'assureur ni le régulateur, crée une responsabilité : s'il est erroné et qu'il a servi à refuser un contrat, qui est responsable ? Techniquement, un score se fait *reconstruire* ou *refuser* : l'actuaire qui reçoit un score non transparent finit par ignorer la boîte noire ou par la réimplémenter.

La conséquence architecturale est forte : puisque le score disparaît, le champ `resolution` devient *le seul signal de qualité* du contrat. Chaque objet aléa doit dire à quel niveau il a été établi.

2.3.2 Un niveau de résolution sur chaque fait

Invariant produit

Chaque objet aléa du contrat porte `source`, `resolution` et les champs d'attribution exigés par la Licence Ouverte 2.0. Un objet aléa sans ces champs échoue la validation du schéma.

Trois valeurs de résolution existent, et elles ne sont pas interchangeables :

Table 2.1. Les trois niveaux de résolution et leur signification exacte.

Valeur	Origine	Signification
`per-building`	Test géométrique WFS	Le point a été testé contre un polygone source. L'affirmation porte réellement sur ce bâtiment.
`commune-level`	Décret ou arrêté	Le fait est communal *par nature* (zonage sismique, classement radon). Ce n'est pas une dégradation : c'est la résolution maximale qui existe.
`commune-level-estimate`	Repli, ou RGA	Nous n'avons pas pu établir le fait au niveau du bâtiment. Le fait est une estimation communale. À traiter avec prudence.

La distinction entre les deux derniers cas est essentielle. Un aléa sismique `commune-level` est *correct* : le zonage est décrétal, il n'existe pas de version par bâtiment. Un aléa inondation `commune-level-estimate` signale au contraire que la source n'a pas répondu et que l'information est dégradée. Confondre les deux ferait passer une panne pour un fait.

2.3.3 Passage verbatim de la BDNB

Invariant produit

Le contrat transmet les attributs BDNB sans suppression ni renommage silencieux. Un champ indisponible est explicitement `null`, avec l'échec enregistré dans la liste d'erreurs — jamais omis.

C'est une conséquence directe du profil actuaire : s'il doit filtrer lui-même, il lui faut l'intégralité du champ. La curation — quels attributs montrer dans le panneau latéral — se fait uniquement à la couche de présentation.

2.3.4 Une seule forme canonique

Invariant produit

Il existe exactement un schéma de réponse par ressource. Deux formes vivantes pour la même ressource constituent un défaut.

Cette règle a une histoire : au cours du projet, deux pipelines concurrents produisaient deux JSON différents pour la même adresse. L'un était utilisé par l'interface, l'autre par le lot. Les divergences silencieuses entre les deux sont exactement le genre de bug qui détruit la confiance dans un contrat de données. La règle a été formalisée en question ouverte (OQ-1) puis tranchée.

2.3.5 Le lot est N fois l'unitaire

Invariant produit

`POST /diagnostic/batch` traite chaque adresse à travers le pipeline unitaire identique. Aucune seconde implémentation en masse, aucun chemin de code d'extraction versionnée.

L'intérêt est la garantie de cohérence : il ne peut pas exister de différence entre le résultat d'un appel unitaire et celui du même appel au sein d'un lot, hors horodatage. Cette équivalence est testée.

2.3.6 Honnêteté sur le RGA

Invariant produit

Tant qu'aucune source vectorielle n'est câblée, le retrait-gonflement des argiles (RGA) est livré avec `resolution: "commune-level-estimate"` et rien ne peut revendiquer une résolution par bâtiment pour cet aléa.

Le RGA est un motif majeur de sinistres en France. La tentation d'afficher un résultat par bâtiment est donc forte. Mais la donnée disponible au moment du stage était communale. Le choix a été de l'annoncer plutôt que de la surinterpréter.

2.3.7 Pas de code dormant

Invariant produit

Les fonctionnalités retirées sont supprimées, non commentées ou laissées non référencées. L'historique Git est l'archive.

Cette règle est inconfortable mais saine : elle interdit de conserver du code « au cas où », qui est la principale source d'ambiguïté dans un dépôt. Elle a conduit à supprimer des milliers de lignes — connecteurs, schémas, composants d'interface — et à assumer que Git garderait la trace.

2.4 Synthèse

Table 2.2. Décisions métier verrouillées lors de la phase de cadrage, et leur traduction en règle technique.

#	Décision	Conséquence technique
1	L'acheteur est l'assureur français, pilote au poste de souscription	Interface orientée dossier unique, pas portefeuille
2	Le livrable principal est un contrat JSON, pas une interface	Schéma Pydantic versionné comme artefact central
3	La jointure WFS × BDNB est le périmètre ; Copernicus et Open-Meteo hors v1	Suppression des connecteurs climatiques de la version 1
4	La transaction est l'adresse unique	`POST /diagnostic/adresse` comme primitive, le lot en dérivé
5	13 aléas, certification sur 4 d'entre eux	Carte de couches en configuration, pas en logique
6	RGA étiqueté communal	Résolution forcée au niveau du schéma
7	Sismique et radon communaux par nature	Résolution permanente `commune-level`
8	BDNB verbatim	Test de conformité clé par clé
9	Score composite mort	Schéma qui refuse la sérialisation de champs de score
10	Bandes D03 en présentation seulement	Coloration côté frontend, hors contrat
11	Le champ de résolution est le signal de qualité	Tests de dégradation aussi rigoureux que les tests nominaux
12	La carte reste, mais montre la jointure	Empreinte colorée par le test, panneau de vulnérabilité
13	Le géocodage est un composant produit	Tests dédiés : arrondissements, seuil de confiance, saturation

CHAPITRE 3

Analyse de l'existant et recentrage

Ce chapitre raconte la décision la plus importante du stage, et la moins confortable : supprimer une grande partie de ce qui avait été construit.

3.1 État de départ : un prototype fonctionnel

Au début du stage, le dépôt contenait déjà une plateforme climatique pour l'immobilier français. Elle interrogeait quatre sources publiques — Géorisques, BDNB, Copernicus, Open-Meteo — et présentait les résultats dans un tableau de bord. Techniquement, cela fonctionnait : les connecteurs répondaient, les données s'affichaient, une couche de notation produisait un score, une visualisation 3D extrudait les bâtiments.

La couche géométrique la plus difficile était même déjà écrite : un parseur GML 3.2 fait main et un test point-in-polygon par lancer de rayon, sans dépendance à `shapely`.

3.2 Le diagnostic : quatre symptômes, une cause

En auditant le prototype, quatre problèmes sont apparus.

3.2.1 La redondance avec les portails sources

C'est le problème central. Pour chaque aléa, le prototype affichait ce que Géorisques affiche déjà, à la même résolution, gratuitement. Pour chaque bâtiment, il affichait ce que la BDNB affiche déjà. Le test appliqué à chaque fonctionnalité a été formulé ainsi :

> *L'acheteur pourrait-il obtenir ceci directement depuis la source ?*

Si la réponse est oui, la fonctionnalité ne justifie pas son existence. Un agrégateur qui reproduit ses sources n'apporte pas de valeur : il apporte une dépendance supplémentaire.

3.2.2 Le score composite

Le prototype produisait un score global par bâtiment, agrégé à partir des aléas. C'est intuitivement satisfaisant — un chiffre, facile à comparer, facile à afficher. Mais c'est précisément ce que le métier refuse (chapitre 2) :

— un score tiers engage la responsabilité de son producteur ;
— un score opaque est soit ignoré, soit reconstruit par l'actuaire ;
— un score mélange des aléas non comparables (inondation et sismicité ne se compensent pas) ;
— sous Solvabilité II, une décision doit être justifiable par des faits, pas par un indice.

3.2.3 Le tableau de bord climatique

Copernicus et Open-Meteo alimentaient un panneau de variables climatiques. Le problème n'était pas la donnée, mais sa forme : les valeurs exposées étaient un *indice d'exposition de 0 à 100*, dérivé, et non des variables climatiques en unités réelles (jours de canicule par an, degrés Celsius, mètres

par seconde). Un indice 0–100 n'est pas une donnée climatique : c'est un jugement déguisé. Il tombait sous le coup de la règle sur les scores.

3.2.4 La décoration 3D

Les extrusions 3D des bâtiments n'apportaient aucune information : elles montraient une forme sans dire si le bâtiment était exposé. C'était un coût de performance et de complexité pour zéro contenu.

3.2.5 La cause commune

Ces quatre symptômes ont une racine unique : **le produit n'était pas organisé autour de la seule chose qu'il faisait et que personne d'autre ne faisait** — la jointure entre exposition et vulnérabilité, à l'échelle du bâtiment. L'effort était réparti sur quatre connecteurs, une couche de notation, un tableau de bord et de la 3D, plutôt que concentré sur cette jointure.

3.3 La décision de recentrage

Le 25 août 2026, une session de cadrage a produit une réorientation complète, documentée dans `docs/workflow/concept.md` et `01-brainstorm-notes.md`. Elle tient en cinq points.

1. **Le livrable est un contrat de données.** L'interface devient un outil de vente et de démonstration, pas le produit.
2. **Le score composite est mort.** Faits et provenance uniquement.
3. **Le périmètre de la v1 est la jointure Géorisques × BDNB.** Copernicus et Open-Meteo n'ont pas d'usage assureur en v1.
4. **Le champ de résolution devient le signal de qualité.** Puisque le score disparaît, c'est lui qui porte l'intégrité du contrat.
5. **Le code mort est supprimé, pas mis en sommeil.**

3.4 L'exécution : supprimer

La conséquence la plus visible du recentrage est une phase de suppression.

Table 3.1. Suppressions de la phase de recentrage, par catégorie.

Catégorie	Éléments supprimés
Connecteurs backend	`copernicus.py`, `era5_land.py`, `flood.py`, `fire_danger.py`, `heat_stress.py`, `wind_risk.py`, `seasonal.py`
Routes backend	`climate.py`, routes `/diagnostic/copernicus/*`
Schémas	`zone_assessment.py`, schéma `RecommandationsIA` et le champ `recommandations`
Scoring	`risk_model.py` et toutes ses références, `app/scoring/`
Dépendances	`cdsapi`, `xarray`, `netCDF4` retirés de `requirements.txt`
Tests	`test_climate_dashboard.py`, `test_copernicus_*.py` (4), `test_trajectoire_brute.py`
Composants frontend	`CopernicusPanel.tsx`, `CopernicusStatusBanner.tsx`, répertoire `jumeau/` (moteur de scène, LIDAR, recommandations), `Artisans.tsx`
Styles	`copernicus.css`, `jumeau.css`, `artisans.css`

Deux règles de méthode ont été respectées pendant ces suppressions. D'abord, elles constituent un *change set dédié* : on ne supprime pas du code au milieu d'une tâche de fonctionnalité, car cela rend la revue impossible. Ensuite, le critère n'était pas « c'est ancien » mais « c'est inatteignable » : avant de supprimer, on vérifie qu'aucune route, aucun import, aucune référence ne pointe vers l'élément. Un code partagé et vivant, comme le module de recommandations du répertoire `jumeau/`, a été *déplacé* plutôt que supprimé.

3.5 Le compromis assumé : ce qui n'a pas été supprimé

Deux éléments ont été conservés malgré la logique de recentrage, et il faut le dire.

La carte.
Elle n'est pas de la décoration : le souscripteur fait du jugement spatial. Mais elle a été ré-orientée : au lieu d'afficher les couches sources, elle affiche la jointure. L'empreinte du bâtiment est colorée par le résultat du test géométrique, accompagnée d'une fiche de vulnérabilité.

Les projections climatiques.
La direction produit initiale les excluait de la v1, mais la non-stationnarité (chapitre 2) est l'argument de fond du produit. Les projections sont donc revenues, sous une forme différente : non plus comme un tableau de bord de variables, mais comme une extension honnête du contrat, avec de vraies unités et une étiquette de résolution.

3.6 Enseignement

La leçon de ce chapitre est méthodologique. Un projet technique n'est pas « bien avancé » parce qu'il contient beaucoup de code. Il est bien avancé quand il devient difficile de lui retirer quelque chose sans le casser. Le recentrage a réduit le périmètre fonctionnel de manière significative et augmenté la valeur du livrable — parce que la valeur n'était pas dans le nombre de fonctionnalités, mais dans l'exactitude de la jointure et dans la traçabilité de chaque fait.

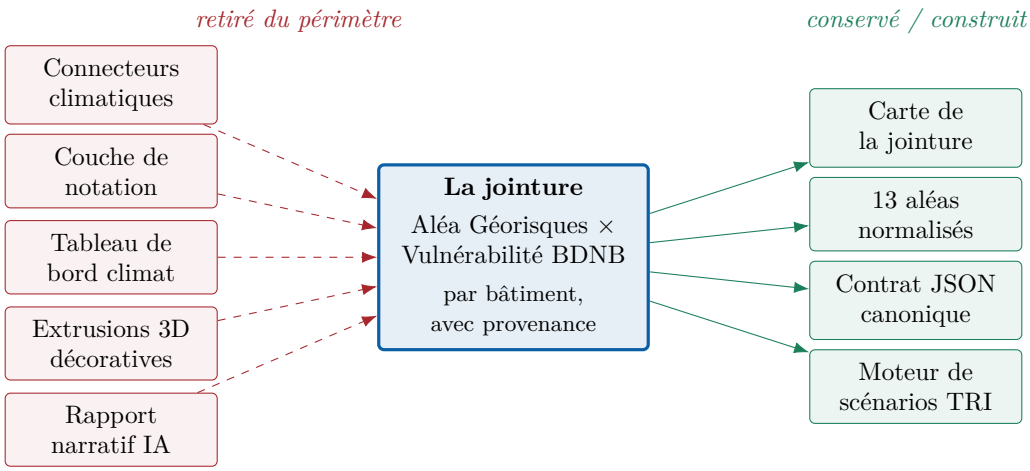


Figure 3.1. Le recentrage en une image. Tout ce qui ne sert pas la jointure aléa × vulnérabilité est retiré du périmètre ou relégué à la couche de présentation.

CHAPITRE 4

Méthodologie d'ingénierie

Un stage court impose une discipline : avec une durée limitée, la principale cause d'échec n'est pas la difficulté technique, c'est la dispersion. Ce chapitre décrit le cadre méthodologique mis en place, inspiré du développement piloté par les spécifications (*spec-driven development*), et les raisons pour lesquelles il a été nécessaire.

4.1 Le problème que la méthode résout

Le projet a une particularité : une part importante du travail a été réalisée en collaboration avec des agents de programmation assistée par IA. Ce mode de travail est puissant pour la production de code, mais il introduit un risque spécifique : l'agent qui reprend le travail ne connaît pas les décisions antérieures, et peut *défaire* un choix de conception sans s'en rendre compte.

Deux exemples réels illustrent ce risque :

— Un agent a réintroduit un niveau d'eau dépendant de la météo du jour. Conséquence : par temps sec, tous les scénarios affichaient 0 m, ce qui se lit comme une panne. Le comportement avait pourtant été explicitement corrigé.
— Un agent a utilisé une boîte englobante à l'échelle du quartier pour interroger le WFS. Mesuré à Nantes : la boîte se remplissait de polygones situés à 2 km, et le polygone contenant réellement l'adresse n'était jamais renvoyé dans la page de résultats. Résultat : « hors TRI » sur un quai inondable.

Dans les deux cas, le code *fonctionnait* au sens où il s'exécutait sans erreur. Le défaut était sémantique. Une suite de tests ne suffit pas si les tests eux-mêmes ne sont pas ancrés dans des décisions explicites.

4.2 Le triptyque documentaire

Trois documents structurent le projet, dans un ordre imposé.

4.2.1 La constitution

La *constitution* (`constitution.md`) énonce les règles non négociables. Chaque règle porte une étiquette d'application :

Table 4.1. Étiquettes d'application des règles constitutionnelles.

Étiquette	Mécanisme	Exemple
`[CI]`	Vérification automatisée	Porte de recherche qui échoue si `niveau`, `score` ou `recommandations` apparaissent dans un schéma émis
`[test]`	Assertion dans la suite	Le schéma refuse la sérialisation d'un objet portant un score, même si cet objet en contient un en interne
`[review]`	Point de contrôle humain	Aucune nouvelle dépendance sans décision enregistrée dans le dépôt

La règle de révision est explicite : *pour changer une règle, on amende la constitution avant le changement qui la violerait*. Cela force la discussion à se tenir au niveau du principe, pas au niveau du correctif.

4.2.2 La spécification

La spécification (`docs/workflow/spec.md`) décrit le produit en exigences fonctionnelles numérotées, FR-01 à FR-28, chacune assortie de critères d'acceptation vérifiables. Par exemple :

FR-03
Rejeter les géocodages de faible confiance avant tout appel aval : score $< 0{,}4 \rightarrow$ HTTP 422 `adresse_ambigue` avec `label_propose`. *Critère :* une requête ambiguë renvoie 422 et aucun appel BDNB ou Géorisques n'a lieu (vérifié par compteurs de mocks).

FR-14
Ne jamais inférer une exposition depuis une source en échec : `present=true` uniquement depuis un test réussi, `false` depuis un test négatif, `null` si inconnu.

FR-26
Cache de résultats en mémoire, clé sur le triplet de géocodage normalisé (libellé + latitude/longitude arrondies à 5 décimales ≈ 1 m + code INSEE) et sur la version de schéma.

Les critères d'acceptation sont rédigés de manière à être directement traduisibles en tests. C'est volontaire : une exigence sans test associé est un souhait, pas une exigence.

4.2.3 Le plan

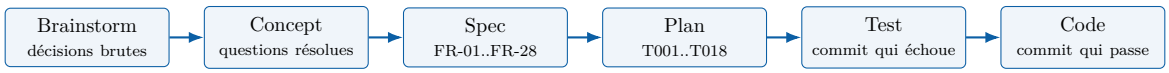
Le plan (`docs/workflow/plan.md`) découpe la spécification en tâches T001 à T018, chacune avec ses dépendances, son estimation, ses points de contact constitutionnels, ses critères d'acceptation et — point important — *les tests à écrire en premier*.

```
### T008 - Certification known-answer suite (v1 sign-off gate)
- Depends on: T004, T007
- Spec reference: FR-11
- Estimate: 4-8 h (fixture collection dominates)
- Acceptance criteria:
  - Recorded fixtures: known-inside and known-outside address per certified
    hazard (inondation/PPRI, mouvement_terrain, feu_foret, avalanche)
  - Inside -> present=true + resolution="per-building"; outside -> present=false
- Tests to write first:
  - test_certified_inside_<hazard> x4
  - test_certified_outside_<hazard> x4
```

Listing 4.1. Extrait du plan : la tâche T008, porte de certification de la v1.

4.3 Le flux de travail

Le flux impose un ordre qui n'est pas négociable :



Deux règles de ce flux méritent d'être soulignées.

1. **Un changement de contrat passe d'abord par la constitution, puis par la spécification — jamais l'inverse.** Si une fonctionnalité nécessite de violer un invariant, ce n'est pas la fonctionnalité qu'on ajuste en premier, c'est le principe.

2. **Les tests existants ne sont jamais modifiés pendant une tâche de fonctionnalité.** Un test jugé erroné est signalé et corrigé dans un change set séparé. Sans cette règle, il est toujours possible de faire passer un test en le réécrivant — ce qui vide la suite de son sens.

4.4 Convention de commit

Les messages suivent une convention préfixée par le domaine : `feat(etape2):`, `fix(flood):`, `test(SCN-xxx):`, `docs:`, `chore:`. Pour toute fonctionnalité, deux commits : un commit `test(TXXX)` qui échoue, puis un commit `feat(TXXX)` qui passe. Cette contrainte rend visible, dans l'historique, que le test a bien précédé le code.

4.5 Décisions documentées

Les décisions qui poseraient problème à un lecteur futur font l'objet de documents courts. Trois exemples du projet :

Table 4.2. Exemples de décisions documentées et leur motivation.

Décision	Document et motivation
Suppression du score composite	`concept.md`, OQ-1 : le score engage la responsabilité et se fait reconstruire ou refuser par l'actuaire
RGA communal	`concept.md`, OQ-2 : la source vectorielle n'est pas câblée, annoncer plutôt que surinterpréter
Licences commerciales	`concept.md`, OQ-3 : Géorisques et BDNB sont en Licence Ouverte 2.0, la réutilisation commerciale est permise, l'obligation d'attribution coïncide avec la conception de provenance
Mode VFX	`constitution.md` §2.1, amendement du 14 septembre : autoriser un rendu cinématique à condition qu'il soit étiqueté non contractuel et strictement confiné à la présentation

4.6 Outils

Table 4.3. Outils et pratiques du projet.

Domaine	Choix
Gestion de version	Git, une tâche = une branche = une PR, messages préfixés
Backend	Python 3.11+, FastAPI, Pydantic v2, `httpx` en asynchrone, `uvicorn`
Frontend	TypeScript, React, Vite, Mapbox GL JS, Vitest
Tests	`pytest` + `pytest-asyncio` (backend), Vitest (frontend)
Lint / format	`ruff` (backend), `eslint` + `tsc -b` (frontend)
Déploiement	Vercel (mono-projet : statique + fonction Python serverless)
Documentation	Markdown dans le dépôt, lu par les humains et par les agents

4.7 Apport de la méthode

Le bénéfice concret de cette organisation a été de rendre les décisions *explicites et opposables*. Lorsqu'un comportement a été corrigé, la correction est inscrite à la fois dans la constitution

(le principe), dans la spécification (l'exigence), dans le plan (la tâche) et dans la suite de tests (l'assertion). Un agent — ou un humain — qui reprend le code trouve la règle avant de trouver le piège.

C'est ce qui a permis de stabiliser un projet dont une part du code a été écrite en collaboration avec des agents, sans que le produit ne dérive.

CHAPITRE 5

Les sources de données et leur intégration

Typhoon n'invente aucune donnée : il assemble des sources publiques. La qualité du produit dépend donc entièrement de la qualité de cette intégration. Ce chapitre décrit chaque source, ce qu'elle apporte, ses contraintes juridiques et techniques, et les pièges rencontrés.

5.1 Vue d'ensemble

Table 5.1. Les sources de données intégrées, leur rôle et leur niveau de résolution.

Source	Rôle	Accès	Résolution
Géorisques (WFS)	Périmètres d'aléas	OGC WFS 2.0, GML 3.2	`per-building` par test point-in-polygon
Géorisques (REST)	Statut communal, arrêtés CatNat	JSON	`commune-level`
BDNB (CSTB)	Vulnérabilité du bâtiment	REST JSON	`per-building` (identifiant bâtiment)
Géoplateforme (IGN)	Géocodage	REST JSON	Point + code INSEE
Classes TRI	Profondeur d'eau réglementaire	REST JSON	Par point, par classe de retour
Sandre	Tracé hydrographique	REST JSON	Tracé
Open-Meteo	Pluie horaire, débit GloFAS	REST JSON	Point, horaire
Panoramax	Photographie terrain	REST JSON	Point, orientée
Copernicus CDS	Projections climatiques 2050/2100	API CDS	Grille ≈ 10 km
Mistral	Rédaction du rapport	REST	— (présentation)

5.2 Géorisques : le cœur de la jointure

5.2.1 Ce que la source fournit

Géorisques expose le référentiel des aléas réglementaires français. Il sert deux interfaces distinctes, et cette dualité est structurante :

— une interface **REST** par code INSEE, qui donne le statut communal : « la commune est-elle concernée par un risque inondation ? » ;

— une interface **WFS** (Web Feature Service) qui expose les géométries vectorielles, donc permet de tester un point précis contre un polygone.

5.2.2 Le test point-in-polygon

Le WFS renvoie du GML 3.2, un dialecte XML volumineux. Le choix a été d'écrire le parseur et le test géométrique à la main, sans dépendance à `shapely`. Deux raisons : éviter une dépendance de compilation lourde dans un contexte serverless, et garder la maîtrise complète des cas limites.

Le test utilise un *lancer de rayon* (*ray casting*) : on compte les intersections d'un rayon partant du point avec les arêtes du polygone. Un nombre impair signifie que le point est à l'intérieur. Le

premier anneau d'un polygone est l'anneau extérieur ; les suivants sont des trous, qu'il faut donc soustraire.

```
def point_in_polygon(pt: tuple[float, float], polygon: Any) -> bool:
    """Point (lon, lat) dans un Polygon / MultiPolygon GML.

    L'anneau exterieur est le premier ; les suivants sont des trous.
    """

def geometry_intersects_point(
    geom: dict[str, Any],
    pt: tuple[float, float],
    radius_m: float = PROXIMITY_RADIUS_M,
) -> bool:
    """Le point (lon, lat) touche-t-il la geometrie ?

    Polygones -> point-in-polygon strict. Points/lignes -> test de proximite
    (une canalisation a 30 m est un risque pertinent meme si le batiment
    n'est pas dessus).
    """
```

Listing 5.1. Signature du test géométrique et gestion des géométries non polygonales (`backend/app/connectors/georisques_wfs.py`).

5.2.3 Les treize aléas

Le référentiel est normalisé en treize objets aléa, chacun portant les mêmes champs obligatoires :

`icpe`, `inondation`, `sismicite`, `mouvement_terrain`, `radon`, `rga`, `cavite`, `feu_foret`, `avalanche`, `canalisations`, `vent_cyclonique`, `ppr`, `ssp`

5.2.4 La ventilation par type de PPR

Un piège est apparu ici. L'agrégat `ppr` répond à la question « un plan de prévention quelconque touche-t-il ce point ? ». C'est insuffisant pour un travail par péril : un bâtiment peut être dans un périmètre PPR sismique sans être dans un périmètre PPR inondation, et l'agrégat ne fait pas la différence.

Une table de correspondance a donc été ajoutée, qui ventile les huit couches PPR par type de péril :

```
# Ventilation des 8 couches PPR de WFS_LAYER_MAP["ppr"] par type de peril.
# L'agregat "ppr" ci-dessus repond "un PPR quelconque touche-t-il ce point ?"
# -- utile pour une carte generale, mais pas pour un peril precis.
PPR_TYPE_LAYERS: dict[str, list[str]] = {
    "inondation":        ["ms:PPRN_PERIMETRE_INOND", "ms:PPRN_PERIMETRE_SUBMAR"],
    "mouvement_terrain": ["ms:PPRN_PERIMETRE_MVT"],
    "seisme":            ["ms:PPRN_PERIMETRE_SEISME"],
    "avalanche":         ["ms:PPRN_PERIMETRE_AVALANCHE"],
    ...
}
```

Listing 5.2. Ventilation des couches PPR par type de péril (`georisques_wfs.py`).

5.2.5 Le piège de la boîte englobante

C'est le bug le plus instructif du projet. Pour interroger un WFS, il faut fournir une *boîte englobante* (BBOX). Le choix naturel est de prendre une boîte « raisonnable », à l'échelle d'un quartier.

Erreur. Le WFS applique un plafond au nombre de géométries renvoyées (paramètre `count`). Mesuré à Nantes : une boîte large se remplit de polygones situés à 2 km, et le polygone contenant réellement l'adresse *n'arrive jamais dans la page de résultats*. Le test conclut alors « hors TRI » sur un quai inondable.

La correction a été de réduire la boîte à l'échelle du point : environ 55 m de marge, soit `POINT_MARGIN_DEG`. Tout polygone contenant le point coupe nécessairement une boîte centrée sur lui : élargir n'ajoute aucune information et fait perdre des réponses.

Honnêteté de la donnée

Un résultat faux n'est pas toujours le signe d'un bug visible. Ici, le service répondait correctement, sans erreur, avec une information fausse. C'est le scénario le plus dangereux pour un produit de données : la panne silencieuse.

5.2.6 Le piège du client HTTP partagé

Autre découverte contre-intuitive : le WFS Géorisques exige un client `httpx.AsyncClient` *dédié*. La passerelle lie la connexion persistante (*keep-alive*) à un backend particulier. Mélanger les appels REST et WFS sur le même client provoque des 404 en cascade — sans rapport apparent avec la cause. Cette contrainte est documentée comme invariant et vérifiée par un test d'intégration qui enchaîne REST puis WFS.

5.2.7 Une réponse vide est ambiguë

Une réponse WFS vide peut signifier deux choses : « aucun périmètre ne contient ce point » (fait) ou « le serveur a hoqueté » (panne). La distinction est essentielle pour le produit. Le connecteur rejoue donc la requête une fois avant de conclure à une absence. C'est un compromis explicite : un surcoût réseau en échange d'une réduction des faux négatifs.

5.2.8 Les arrondissements

Paris, Lyon et Marseille ont un découpage administratif particulier : les arrondissements ont leur propre code INSEE (75101 à 75120 pour Paris), mais les endpoints communaux attendent le code de la commune parente (75056). Sans retraduction, la jointure échoue silencieusement. La correspondance est implémentée et testée.

5.3 La BDNB : la vulnérabilité

La Base de Données Nationale des Bâtiments, produite par le CSTB, décrit chaque bâtiment français. Le connecteur récupère environ 139 champs : usage, hauteur, emprise au sol, matériaux de construction, année de construction, performance énergétique, et des indicateurs de fiabilité de l'adressage.

Deux points de conception :

1. **Passage verbatim.** Aucun renommage, aucun filtrage. Un test de conformité compare l'ensemble des clés reçues et l'ensemble des clés émises : ils doivent être identiques.
2. **Ambiguïté d'adresse.** Plusieurs bâtiments peuvent partager une adresse. La première ligne renvoyée est traitée comme le bâtiment principal ; les autres sont conservées verbatim dans `autres_batiments_meme_adresse[]` — jamais supprimées. Un test couvre le cas de trois bâtiments à une même adresse.

5.4 Les classes TRI : la profondeur d'eau réglementaire

Pour la mise en situation sur la carte (chapitre 7), il faut une profondeur d'eau. La source utilisée est le **Territoire à Risque d'Inondation** (TRI), issu de la Directive Inondation. Le TRI classe l'exposition en quatre niveaux correspondant à des périodes de retour :

Table 5.2. Les classes TRI officielles et leur signification.

Classe	Période de retour	Lecture
`frequent`	≈ 10 ans	Événement courant
`moyen`	≈ 100 ans	Événement de référence
`extreme`	≈ 500 ans	Événement rare
`faible`	au-delà	Exposition résiduelle

Le point crucial : ces classes et leurs profondeurs sont des **facts réglementaires**. Elles ne sont jamais inventées ni interpolées. Si le point n'est pas cartographié, le produit le dit — il ne devine pas une hauteur.

5.5 Les sources complémentaires

Géocodage — Géoplateforme (IGN).
L'adresse saisie est convertie en coordonnées, code INSEE, code postal et libellé normalisé, avec un score de confiance. Le géocodage est traité comme un composant produit, pas comme une utilité : gestion des arrondissements, rejet des scores faibles, gestion de la saturation (HTTP 429 avec en-tête `retry-after`).

Hydrographie — Sandre.
Le tracé des cours d'eau, utilisé pour l'animation de suivi de l'eau sur la carte.

Météo — Open-Meteo.
Pluie horaire et débit GloFAS. Utilisée pour la *forme* de la montée des eaux, jamais pour son niveau (voir chapitre 7).

Photographie — Panoramax.
Une photo de terrain orientée sur le point, utile au souscripteur pour un contrôle visuel rapide.

Copernicus CDS.
Les projections climatiques à 2050 et 2100, en vraies unités (jours de canicule par an, degrés, mètres par seconde), pour les deux scénarios RCP 4.5 et RCP 8.5.

Mistral.
La rédaction du rapport narratif. Strictement confinée à la couche de présentation : elle ne produit aucun fait et n'entre jamais dans le contrat canonique.

5.6 Contraintes juridiques et quotas

5.6.1 Licence Ouverte 2.0

Géorisques et la BDNB sont distribués sous **Licence Ouverte 2.0** (Etalab). La réutilisation commerciale est explicitement autorisée, sans limitation de durée ni de territoire. La seule obligation est l'**attribution** : nom de la source et date de dernière mise à jour.

Cette obligation coïncide exactement avec la conception de provenance du produit : la conformité juridique devient une fonctionnalité, pas une charge. Chaque bloc de provenance du contrat porte une chaîne d'attribution du type :

```
Source Géorisques (BRGM / MTE) - données à jour au 2026-07-01
```

Le millésime est stocké en configuration par source et mis à jour lorsque le fournisseur publie une nouvelle version.

5.6.2 Quotas

Les quotas des fournisseurs sont respectés côté client, car les dépasser signifie une coupure de service :

Table 5.3. Quotas amont et mécanismes de protection implémentés.

Fournisseur	Quota	Protection
BDNB	120 req/min/IP, 10 000 req/mois (offre gratuite)	Limitation de débit côté client + compteur budgétaire mensuel
Géoplateforme	50 req/s/IP	Garde de rafale + gestion du 429 avec réattente unique

Un garde budgétaire mensuel protège l'allocation BDNB : lorsque le compteur est épuisé, les soumissions de lots sont rejetées avec un HTTP 429 `budget_epuise`, tandis que les requêtes unitaires interactives continuent de fonctionner. Augmenter le débit réel est une action commerciale (souscrire à l'offre supérieure), pas une décision de code.

5.7 Le cache

Un cache en mémoire à durée de vie limitée (TTL, 24 h par défaut, configurable) évite de re-interroger les sources pour une même adresse. La clé est construite sur le triplet de géocodage normalisé et sur la version de schéma :

```
libellé normalisé + latitude/longitude arrondies à 5 décimales + code INSEE +
schema_version
```

L'arrondi à 5 décimales correspond à environ 1 m : deux adresses distinctes ne partagent donc pas une clé par accident. Un point d'honnêteté : un résultat servi depuis le cache conserve ses horodatages d'origine. Il indique la date de la collecte réelle, pas la date de la consultation.

Honnêteté de la donnée

Sur l'instance serverless déployée, ce cache ne persiste pas — le système de fichiers de la fonction est éphémère. Chaque rapport narratif consomme donc un appel au modèle, et la prose varie légèrement entre deux requêtes identiques. En local, le cache rejoue le rapport à l'identique. Cette différence est documentée dans le README plutôt que masquée.

CHAPITRE 6

Architecture et contrat de données

6.1 Le principe : une seule primitive

L'architecture repose sur une idée simple : il existe *une* transaction produit, et toutes les autres surfaces n'en sont que des rendus ou des répétitions.

> **La transaction unitaire** `POST /diagnostic/adresse` prend une adresse en entrée et renvoie un enregistrement JSON canonique. La carte l'affiche, le lot la répète, l'export PDF la met en page. Aucune autre surface n'a le droit de posséder sa propre logique métier.

Cette contrainte a une conséquence forte : toute amélioration de la qualité de la transaction unitaire bénéficie automatiquement à l'ensemble du produit. Elle interdit aussi la classe de bugs la plus coûteuse pour un contrat de données — deux implémentations qui divergent lentement.

6.2 Vue en couches

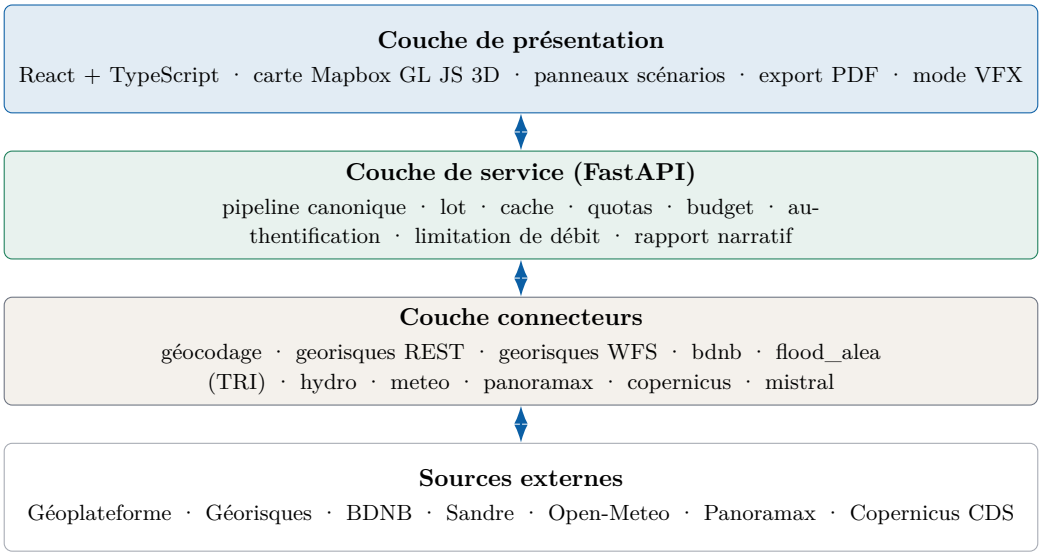


Figure 6.1. Architecture en quatre couches. La règle d'or : aucune couche supérieure n'appelle une source externe directement, et aucun fait ne remonte sans sa provenance.

6.3 Structure du dépôt

```
backend/
  app/api/routes/        diagnostic, flood, hydro, photo, report,
                         geocoding, health
  app/connectors/        bdnb, georisques (REST + WFS), flood_alea (TRI),
                         hydro (Sandre), meteo (Open-Meteo), copernicus,
                         geocoding (BAN), panoramax, mistral
  app/services/          canonical (pipeline unitaire), batch, budget,
```

```
                          limits, report_agent, latency_report
  app/schemas/            diagnostic_record (contrat serialise), flood,
                          hydro, photo, report, building_data
  app/core/               config, logging, api_key, rate_limit
  tests/                  24 modules de test hors-ligne

frontend/
  src/routes/             Zone (/zone), ReportPage, Dashboard, Portfolio,
                          WatchlistPage, Login, Home, Contact, Faq
  src/components/         UnifiedMap (Mapbox 3D), ScenarioPanel, RiskConsole,
                          RiskPanel, FloodConsole, LeftPanels, VfxDisclaimer
  src/zone/               moteur : floodAlea (client TRI), floodSim, damageModel,
                          impactModel, hazardSim, windSim, exposure,
                          hydroRoute, hydroLayer, pdf-export, canonicalAdapter
  src/zone/vfx/           vfxInputs (bus d'entree du mode cinematique)
  src/typhoon/            chrome applicatif, authentification, navigation

docs/workflow/            brainstorm, concept, spec, plan, scenario-plan,
                          vfx-spec, vfx-plan
constitution.md           regles non negociables
AGENTS.md                 conventions et pieges, lu avant toute modification
```

Listing 6.1. Organisation des fichiers du projet (simplifiée).

6.4 Le contrat canonique

C'est l'artefact central du produit. Sa structure complète figure en annexe A ; on en présente ici la logique.

6.4.1 Une adresse, treize aléas, un bâtiment

```
{
  "schema_version": "1.0",
  "adresse": {
    "saisie": "Quai de la Rapee, 75012 Paris",
    "normalisee": "...",
    "citycode": "75056", "postcode": "75012", "city": "Paris",
    "lat": 48.848, "lon": 2.370,
    "geocode_score": 0.95
  },
  "aleas": [                    // 13 objets, referentiel complet
    {
      "code": "inondation",
      "libelle": "Inondation",
      "present": true,          // true | false | null (inconnu)
      "present_commune": true,
      "zonage": "Dans un perimetre PPR inondation",   // texte verbatim
      "hauteur_eau_m": 1.5,
      "per_building": {         // present uniquement si teste geometriquement
        "method": "point-in-polygon",
        "radius_m": null,
        "count": 2
      },
      "source": "georisques",
      "url_detail": "https://www.georisques.gouv.fr/...",
      "erreur": null,
      "resolution": "per-building"
    }
  ],
  "bdnb": {
    "donnees": { /* ~139 champs verbatim */ },
```

```
    "_source": {
      "provider": "BDNB", "url": "https://api.bdnb.io",
      "attribution": "Source BDNB (CSTB) - donnees a jour au <millesime>",
      "recuperee_le": "2026-08-25T10:00:00+00:00"
    }
  },
  "georisques_source": { /* provider, url, attribution, recuperee_le */ },
  "erreurs_partielles": [],
  "genere_le": "2026-08-25T10:00:01+00:00",
  "avertissement": "Ce rapport agrege les donnees publiques Georisques..."
}

// INTERDIT dans ce schema : niveau / bandes DO3, tout score numerique,
// recommandations, copernicus, tout jugement composite. (FR-18, const. §2)
```

Listing 6.2. Squelette du contrat canonique (simplifié).

6.4.2 La sémantique du champ `present`

La valeur `present` est à trois états, et cette trichotomie porte une partie de la valeur du produit :

Table 6.1. Les trois états de `present` et leur signification.

Valeur	Signification	Origine
`true`	Le point est concerné	Un test géométrique réussi, une proximité mesurée, ou un statut communal positif
`false`	Le point a été testé et n'est pas concerné	Un test géométrique négatif, ou un statut communal négatif
`null`	On ne sait pas	La source a échoué. Le champ `erreur` est rempli et une entrée est ajoutée à `erreurs_partielles`

La règle FR-14 interdit formellement d'inférer une valeur booléenne depuis une source en échec. Un test de propriété le vérifie : quel que soit le mode de panne injecté (délai d'attente, erreur 5xx, charge malformée, réponse vide), le résultat est `null` avec une erreur renseignée — jamais `true` ou `false`.

Honnêteté de la donnée

C'est la règle qui distingue un produit de données honnête d'un produit « qui a l'air de marcher ». Un aléa absent parce que la source a échoué ressemble exactement à un aléa absent parce qu'il n'existe pas. Sans la trichotomie, l'utilisateur conclut « pas de risque » là où il devrait lire « je ne sais pas ».

6.4.3 La règle de sérialisation des champs optionnels

Un détail qui a son importance pour un consommateur automatique : les champs spécifiques à un aléa sont *toujours présents* dans le JSON, à `null` lorsqu'ils ne s'appliquent pas. Une clé n'est jamais absente. Cela évite au client d'avoir à distinguer « clé absente » de « valeur nulle », distinction qui est une source classique de bugs d'intégration.

Une exception documentée : l'objet `per_building` n'existe que si le point a effectivement été testé géométriquement. Sa présence est elle-même porteuse d'information.

6.4.4 Le schéma comme garde-fou

L'invariant « pas de score »n'est pas une convention de style : il est imposé par le schéma Pydantic. Un test vérifie qu'un objet contenant temporairement un score en interne se sérialise *sans* les clés interdites. Le schéma refuse activement les champs `niveau`, les scores numériques, `recommandations` et `copernicus`.

6.5 Le pipeline unitaire

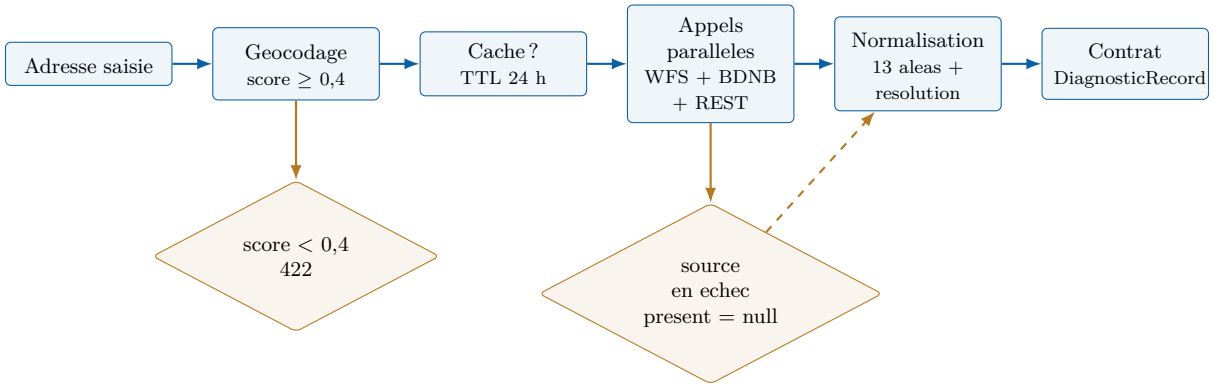


Figure 6.2. Le pipeline unitaire. Les sorties dégradées (en ambre) traversent le même chemin que les sorties nominales : il n'existe pas de code séparé pour les cas d'échec.

6.5.1 Le géocodage d'abord, et comme point de non-retour

Le géocodage est la première étape et constitue un point de non-retour. Sans coordonnées résolues, il n'y a rien à joindre : une adresse introuvable ou trop ambiguë arrête la transaction avec un HTTP 422. C'est le seul cas où le pipeline échoue plutôt que de se dégrader, et c'est logique — la dégradation suppose qu'il existe quelque chose à dégrader.

Le seuil de confiance est fixé à 0,4. En dessous, la réponse propose le libellé le plus proche. Un test vérifie qu'aucun appel BDNB ou Géorisques n'a lieu dans ce cas (compteurs de mocks).

6.5.2 Collecte parallèle

Les sources sont interrogées en parallèle via une structure de collecte sécurisée (`_safe_call`) : chaque appel est isolé, avec son délai d'attente et son traitement d'erreur. L'échec d'une source ne fait jamais échouer la transaction.

6.5.3 Normalisation

La normalisation est l'étape où la valeur du produit se construit : elle transforme les charges brutes hétérogènes en treize objets aléa homogènes, chacun portant sa provenance et son niveau de résolution. C'est également elle qui applique les règles forcées : RGA toujours `commune-level-estimate`, sismique et radon toujours `commune-level`.

6.6 Les routes de l'API

Table 6.2. Surface de l'API backend.

Route	Description
`POST /diagnostic/adresse`	Transaction canonique : adresse → enregistrement complet
`POST /diagnostic/batch`	Soumission d'un lot ($\leq$ 10 000 adresses) ; renvoie un identifiant
`GET /diagnostic/batch/{id}`	Interrogation de l'état du lot
`GET /diagnostic/adresse/rapport-pdf`	Relais vers le PDF officiel Géorisques (commodité souscripteur)
`GET /diagnostic/zone/building(s)`	Fiches BDNB autour du point
`GET /api/flood-alea`	Classes TRI au point (WFS, point-in-polygon)
`GET /api/hydro`, `/api/hydro/extend`	Tracé hydrographique (Sandre)
`GET /api/meteo`	Pluie horaire + débit GloFAS (Open-Meteo)
`GET /api/photo`	Photographie Panoramax et crédit
`GET /api/report`, `/api/report/stream`	Rapport narratif (Mistral, repli sur gabarit si la clé est absente)
`GET /api/geocode/search`	Autocomplétion d'adresse (BAN)
`GET /health`, `/health/detailed`	Sonde de vie (non authentifiée)

6.7 Sécurité

6.7.1 Authentification par clé

L'authentification est volontairement simple : une clé statique transmise dans l'en-tête `X-API-Key`, validée contre une liste configurée par variable d'environnement. Une clé absente produit un 401 ; une clé invalide, un 403. Les routes de santé sont exemptées.

Un choix explicite : **les clés ne sont jamais journalisées**. Un test capture les journaux et vérifie que le matériel de clé est absent. Les adresses, en revanche, peuvent apparaître dans les journaux applicatifs — elles constituent l'interface de requête, et les tracer est nécessaire pour le diagnostic.

6.7.2 Défenses en profondeur

— Tous les appels externes portent un délai d'attente explicite. Aucune attente non bornée n'existe dans le code.

— Une limitation de débit entrante protège l'API Typhoon elle-même, distincte de la conformité aux quotas amont.

— Aucun secret dans le code : toutes les informations d'authentification passent par des variables d'environnement, et le fichier `.env.example` documente chaque variable.

6.8 Le mode dégradé

Le traitement des pannes est conçu comme une fonctionnalité de premier ordre. Le tableau 6.3 résume le comportement attendu dans chaque situation.

Table 6.3. Matrice des erreurs : ce que le client observe dans chaque situation.

Condition	Comportement observable
Adresse introuvable	422 `adresse_non_trouvee`, avant tout appel source
Score de géocodage < 0,4	422 `adresse_ambigue` avec libellé proposé
Géocodeur indisponible	429 honoré une fois via `retry-after`, puis 502 `geocodage_indisponible`
Une couche WFS en échec	Cet aléa : `present=null`, `erreur` renseignée, entrée dans `erreurs_partielles` — réponse 200 partielle
Géorisques entièrement indisponible	Les faits communaux disponibles sont conservés ; les faits géométriques passent à `null`
BDNB indisponible	`bdnb=null` + entrée dans `erreurs_partielles`
Clé absente ou invalide	401 / 403, avant tout traitement
Échec d'un élément de lot	L'élément est signalé individuellement, le lot continue
Exception inattendue	Journalisée avec trace, 502 structuré — jamais de 500 HTML

La règle de conception sous-jacente : *le mode dégradé est testé avec la même rigueur que le chemin nominal*. Une suite de tests dite « chaos » éteint chaque source indépendamment, dans chacun de ses modes de défaillance, et vérifie que la réponse reste un 200 partiel structuré.

CHAPITRE 7

Le moteur de scénarios interactifs

Un contrat JSON est difficile à vendre. Un souscripteur qui voit un JSON comprend la donnée, mais il ne *ressent* pas le risque. C'est la raison d'être de la mise en situation sur la carte : rendre visible, sans mentir, ce que le contrat décrit.

Ce chapitre raconte la construction de ce moteur — et les trois bugs sémantiques qui ont dû être corrigés, chacun révélant une confusion conceptuelle plus profonde qu'un simple défaut de code.

7.1 Objectif et référence

L'objectif est une boucle : choisir une classe de retour → l'eau monte sur le bâtiment sur la carte ; déplacer un curseur horaire → le niveau évolue en direct ; appuyer sur « Play » → l'animation se déroule jusqu'au pic.

La référence visuelle assumée est un projet de démonstration de visualisation d'inondation (FuseLab), connu pour sa capacité à rendre une montée des eaux lisible et crédible. L'enjeu était de reproduire cette lisibilité *sans* enfreindre les invariants du produit : aucune profondeur inventée, aucune affirmation non étiquetée.

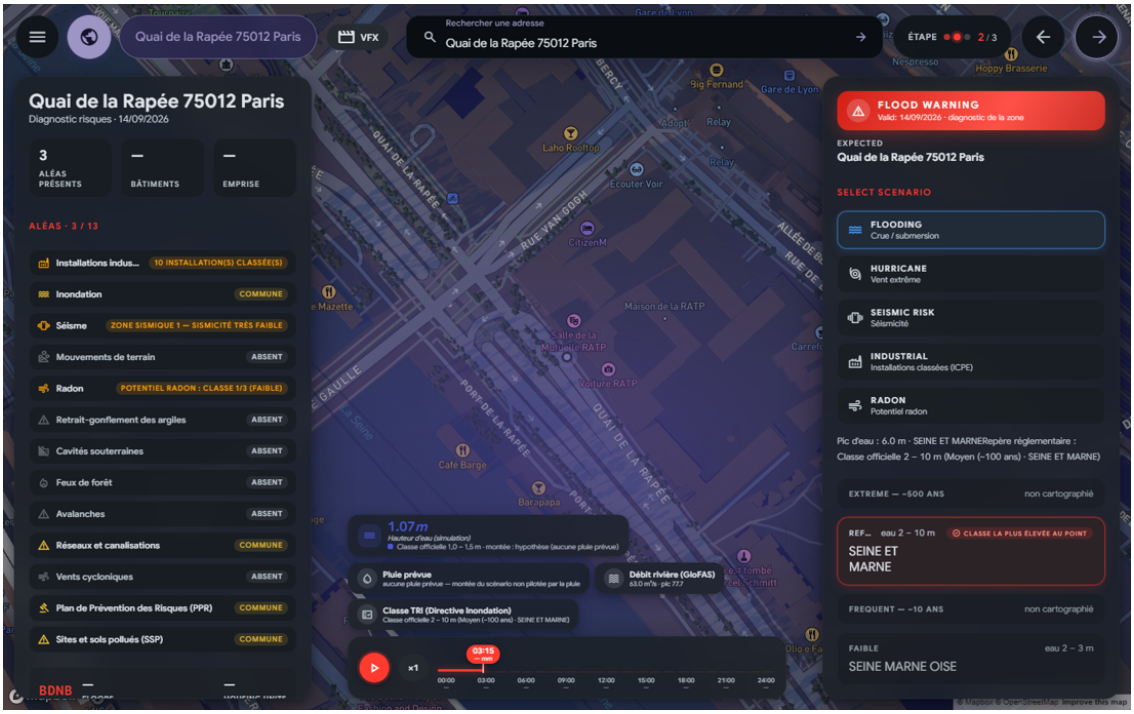


Figure 7.1. Étape 2 — simulation de scénario. À gauche, les classes TRI officielles ; à droite, la timeline horaire et le bouton de lecture. L'eau monte sur le bâtiment par extrusion Mapbox.

7.2 Le bus d'état unique

L'architecture repose sur un bus d'état minimal, partagé par la carte et les panneaux :

`scenarioKey` (classe TRI) · `hazardEvent` (aléa sélectionné) · `riskTimeMin` (heure simulée)

Chaque aléa branche un rendu cartographique distinct — nappe d'eau, cône de feu, couche de vent — mais tous lisent le même moteur. Aucun panneau ne recalcule un niveau de façon indépendante : c'est la garantie que la console et la carte affichent la même profondeur.

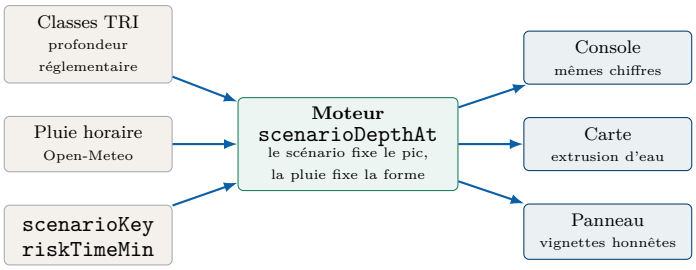


Figure 7.2. Un seul bus d'état, un seul moteur. La carte et les panneaux ne peuvent pas diverger.

7.3 Bug 1 : les clés de scénario ne correspondaient pas

7.3.1 Symptôme

Changer de scénario ne produisait rien : la carte restait sèche.

7.3.2 Cause

Le moteur utilisait des clés de scénario (`extreme`, `moyen`, `frequent`, `faible`) tandis que la couche d'interface utilisait une autre table de correspondance (`direct`, `west`, `east`, `offshore`), vestige d'une première version où les scénarios étaient géographiques. La fonction de résolution recevait donc une clé inconnue et renvoyait `undefined`, silencieusement converti en `null` — donc aucune eau.

7.3.3 Correction

Décision SCN-001 : **les clés du moteur *sont* les clés réglementaires TRI**. L'indirection est supprimée, ou réduite à un alias documenté comme déprécié. Il n'existe plus qu'une seule source de vérité.

7.4 Bug 2 : le seuil de dommages masquait la carte

7.4.1 Symptôme

Même avec une clé correcte, la carte ne peignait rien avant un certain niveau.

7.4.2 Cause

Un seuil unique de 0,3 m servait à la fois à décider « le bâtiment subit-il des dommages ? » et « faut-il peindre de l'eau sur la carte ? ». Or une nappe de 0,1 m est un fait visuel réel, même si elle n'endommage pas les équipements. Le seuil de dommages masquait la réalité physique.

7.4.3 Correction

Décision SCN-002 : **deux seuils, jamais confondus**. Le seuil de 0,3 m gouverne les dommages, les compteurs et les tuiles « inondé ». La carte, elle, peint dès que la profondeur dépasse une valeur plancher de 2 cm (`MAP_MIN_DEPTH_M`), en dessous de laquelle il ne reste qu'un résidu numérique.

Honnêteté de la donnée

Le seuil plancher de 2 cm est documenté explicitement : « à t_0 l'eau est visible » signifie « au-delà de 2 cm ». C'est un détail, mais c'est le genre de détail qui, non documenté, devient un rapport de bug incompréhensible six mois plus tard.

7.5 Bug 3 : « rien ne se passe » par temps sec

7.5.1 Symptôme

Sur une adresse réelle à Paris, après correction des deux bugs précédents, le bouton de lecture restait inerte et changer de scénario ne produisait toujours rien.

7.5.2 Cause — deux causes distinctes

L'investigation sur données réelles a montré deux problèmes indépendants.

Cause A : le niveau dépendait de la météo du jour. Le niveau d'eau était calculé à partir de la prévision de pluie. Or ce jour-là à Paris, le profil de pluie était vide (`dry: true`, cumul 0 mm) : la profondeur valait donc 0 et le bouton était désactivé.

C'est une confusion conceptuelle : *une classe de retour réglementaire n'est pas une prévision météo*. La classe TRI « 100 ans » existe indépendamment du fait qu'il pleuve aujourd'hui.

Cause B : la classe ouverte par défaut n'était pas cartographiée. Le scénario sélectionné par défaut était `extreme`, mais au point considéré seule la classe `faible` était cartographiée. La résolution renvoyait `null`, donc aucune eau.

7.5.3 Corrections

Décision SCN-004a — **le scénario pilote le niveau, la pluie ne pilote que la forme** :

```
scenarioDepthAt(profile, hourIndex, peakM)
```

La classe TRI officielle fixe le pic — c'est un fait réglementaire. La prévision de pluie ne fixe que la *forme* de la courbe. En l'absence de pluie, une rampe divergée documentée prend le relais et l'interface déclare explicitement « montée : hypothèse (aucune pluie prévue) ». L'ancienne fonction dépendante de la météo reste exportée mais marquée dépréciée, avec l'interdiction explicite de l'utiliser comme source du niveau.

Décision SCN-004b — **sélection par défaut = classe la plus intense réellement cartographiée**. Une fonction bascule sur la classe la plus intense cartographiée, une seule fois par diagnostic, pour ne pas annuler un choix explicite de l'utilisateur. Les classes non cartographiées deviennent `disabled` avec une infobulle : un clic sans effet se lit comme un bug.

Invariant produit

Hors TRI ou absence de classe au point → strictement 0 m. Le pic n'est *jamais* deviné. La fonction `scenarioDepthAt` n'exagère que la forme de la courbe, jamais son maximum.

7.6 Le mode VFX et l'isolation des modes

Un second mode de rendu, dit VFX, a été ajouté ultérieurement (chapitre 8). Un point de conception important apparaît ici : le mode VFX consomme *le même* `scenarioDepthAt` que le mode

données. Il est donc impossible que les deux modes affichent deux hauteurs différentes.

De plus, lorsque le mode VFX est actif, la nappe d'eau du mode données est explicitement désactivée (`impact=null`), ce qui évite deux nappes superposées. Cette isolation est vérifiée par un test dédié.

7.7 L'absence n'est pas une panne

Un principe transversal mérite d'être isolé, car il a été formalisé comme invariant (SCN-003) : **l'absence d'un aléa et la panne d'un service sont deux faits distincts**.

Le connecteur TRI porte cinq vérités différentes :

1. chargement en cours ;
2. service indisponible (panne) ;
3. le point est dans un TRI mais aucune classe n'y est définie ;
4. le point est hors TRI ;
5. l'appartenance n'a pas pu être vérifiée.

La formulation est *partagée* entre le panneau, la console et le rapport : une seule fonction produit le texte affiché. Cela évite le scénario classique où trois composants décrivent le même état de trois façons légèrement différentes, ce qui donne à l'utilisateur l'impression d'un système incohérent.

Table 7.1. Les messages distincts pour l'absence d'exposition.

Situation	Message
Hors TRI	« Point non cartographié · Hors zone TRI » — un fait
Dans un TRI, sans classe au point	Message distinct, sans chiffre — un fait réglementaire
Service indisponible	« Service indisponible · la cartographie TRI n'a pas répondu » — une panne

7.8 Les autres aléas

7.8.1 L'ouragan

En mode ouragan, l'eau est forcée à `null` — il n'y a pas d'inondation — et une couche météorologique synchronisée à l'heure simulée affiche le vent et la pluie. La console affiche la rafale et la bande correspondante.

7.8.2 Le feu

Pour l'aléa feu, un cône d'exposition est calculé à partir de la direction réelle du vent et tracé sur la carte.

```
fireConePolygon(meteo, originLonLat) -> GeoJSON Feature
  - cap     = provenance du vent + 180 degres
  - portee  = bornee par FIRE_REACH_MAX_KM = 1.5 km
  - ouverture heritee de fireConeFrom
```

Listing 7.1. Construction du cône d'exposition au feu.

Un écart assumé par rapport au plan initial mérite d'être signalé : la couche devait être une extrusion 3D, elle a été réalisée en remplissage (`fill`) et contour pointillé. La raison est explicite :

un volume 3D sur un cône d'exposition suggérerait une précision qui n'existe pas. Le cône est une zone d'exposition possible, pas une emprise de flammes.

Le cône n'est monté que si l'aléa feu est sélectionné *et* que le diagnostic indique une exposition au feu. Sans météo, aucun cône n'est tracé et un message explicite s'affiche.

7.9 Synthèse des décisions du moteur

Table 7.2. Décisions verrouillées du moteur de scénarios.

Décision	Contenu
SCN-001	Les clés de scénario du moteur sont les clés réglementaires TRI. Pas de table de correspondance locale.
SCN-002	Deux seuils distincts : dommages (0,3 m) et carte (dès 2 cm).
SCN-003	Absence ≠ panne : cinq vérités distinctes, un texte partagé.
SCN-004a	Le scénario pilote le niveau, la pluie ne pilote que la forme.
SCN-004b	Sélection par défaut = classe la plus intense cartographiée, une fois par diagnostic.

Ces cinq décisions sont inscrites dans `AGENTS.md`, le document que tout contributeur — humain ou agent — doit lire avant de modifier le code. Chacune est accompagnée de la formulation « ne jamais réintroduire », avec la raison historique. C'est la réponse méthodologique au problème posé au chapitre 4 : un correctif qui n'est pas documenté comme invariant sera défait.

CHAPITRE 8

Le mode de rendu cinématique (VFX)

Ce chapitre décrit une fonctionnalité volontairement contradictoire avec le reste du projet, et la manière dont la contradiction a été encadrée.

8.1 Le problème : la tension

Le produit est fondé sur l'honnêteté de la donnée : pas de score, pas d'invention, une provenance sur chaque fait, une étiquette de résolution. Une démonstration cinématique — caméra qui vole, eau qui monte avec des reflets, particules, effets — est à l'opposé : elle vise l'impression, pas la preuve.

La tentation était double :

— **commerciale** — une démonstration spectaculaire vend un concept mieux qu'un JSON ;
— **technique** — un rendu 3D temps réel est un objet d'ingénierie intéressant.

Le risque était tout aussi double : introduire un rendu qui *paraît* factuel alors qu'il est illustratif, et laisser ce rendu contaminer le contrat de données.

8.2 La solution : gouverner la contradiction

Plutôt que de refuser la fonctionnalité ou de l'accepter telle quelle, le projet l'a encadrée par un amendement constitutionnel. La constitution a été portée en version 0.2 le 14 septembre 2026, avec un paragraphe §2.1 dédié.

8.2.1 Deux modes obligatoires

Table 8.1. Les deux modes de rendu et leurs contrats respectifs.

	Mode `data`	**Mode `vfx`**
Rendu	Enveloppes honnêtes, extrusion Mapbox sur les empreintes réelles	Couche personnalisée Three.js sur Mapbox, terrain, particules, post-traitement
Défaut	**oui**	non — activé explicitement
Étiquetage	Aucun (le rendu est factuel)	Bandeau permanent « Simulation visuelle — non contractuelle »
Données partielles	Mention d'indisponibilité explicite	Étiquette secondaire « Données partielles — rendu illustratif »
Contrat API	Inchangé	**inchangé**

Invariant produit

Aucune route backend, aucun champ de schéma, aucune charge de lot ne peut transporter de géométrie VFX, de score ou de jugement synthétique. Le mode VFX est strictement confiné à la présentation.

8.2.2 Règles d'entrée

Le mode VFX doit consommer *les mêmes* pilotes que le mode données lorsqu'ils sont disponibles : l'heure simulée, les profils météorologiques, les bandes de profondeur TRI, le trajet hydrographique, la hauteur et l'emprise BDNB. Il peut exagérer l'*apparence* — montée lissée, plan d'eau suivant le terrain, floraison lumineuse, mouvement de caméra — mais il ne peut pas revendiquer une précision hydraulique.

Les replis illustratifs sont autorisés *uniquement* en mode VFX et *uniquement* avec l'étiquette de données partielles.

8.2.3 Le bus d'entrée unique

Un module dédié, `vfxInputs.ts`, est le seul point d'entrée du rendu cinématique. La règle est explicite : le rendu VFX passe par ce module et rien d'autre.

```
vfxWaterLevel({ rainProfile, hourIndex, triPeak, allowFallback })
  -> { levelM, partial, source: 'forecast' | 'fallback' | 'none' }
```

Listing 8.1. Le bus d'entrée du mode VFX et sa règle de repli.

Deux garanties ont été codées dans ce module :

1. Le repli illustratif est **plafonné par la classe TRI** : $\min(\text{triPeak}, 1{,}0\ \text{m})$. Le rendu ne peut donc jamais exagérer au-delà de l'exposition réglementaire réelle.
2. **Sans classe TRI, la source vaut `none` et le niveau vaut 0**, même si le repli est autorisé. On n'invente pas une exposition qui n'existe pas.

Ce point est important : le mode cinématique peut embellir la *forme*, jamais créer un *fait*. Une adresse hors zone inondable reste sèche, même en mode spectaculaire.

Honnêteté de la donnée

Un problème d'incohérence a été détecté et corrigé pendant la conception : la spécification initiale prévoyait une constante de repli de 1,0 m indépendante de la classe TRI. Elle était à la fois incohérente — elle pouvait afficher plus que le risque réel — et inatteignable, puisque le mode données consommait une autre fonction. Elle a été supprimée : le repli ne porte plus que sur la *forme* de la courbe.

8.3 Contraintes d'intégration

8.3.1 Exclusion mutuelle avec la couche météo

La couche météo Windy impose une projection plate (Mercator sans inclinaison). La caméra 3D du mode VFX, elle, exige une inclinaison forte. Les deux sont incompatibles.

Invariant produit

La couche Windy et la caméra 3D du mode VFX sont mutuellement exclusives tant que le mode VFX est actif. Cette exclusivité est vérifiée par un test.

Concrètement : à l'activation du mode VFX, la couche Windy est désactivée et l'inclinaison de la carte est forcée au-delà de 55 degrés. À la sortie, la politique du mode données reprend la main et l'état précédent est restauré.

8.3.2 Le montage et le démontage

Le mode VFX doit s'activer et se désactiver proprement. La contrainte est double : aucune fuite de contexte WebGL au changement de route, et aucun doublon de couche en cas de bascule répétée.

Invariant produit

La couche VFX se monte et se démonte proprement lors de la bascule de mode. Aucune fuite de contexte WebGL au changement de route. Objectif de performance : au moins 30 images par seconde sur un ordinateur portable de milieu de gamme au niveau de zoom d'un bâtiment avec le plan d'eau actif, avec dégradation gracieuse — le post-traitement est désactivé en premier.

8.4 Étiquetage

L'étiquetage n'est pas cosmétique : c'est la condition qui rend la fonctionnalité acceptable.

— Le bandeau « **Simulation visuelle — non contractuelle** » est affiché en permanence en mode VFX.

— Une étiquette secondaire « **Données partielles — rendu illustratif** » apparaît lorsque les entrées réelles sont incomplètes.

— Le mode VFX **ne doit pas réutiliser le vocabulaire du contrat** (`resolution`, `present`, libellés de bandes) comme s'il s'agissait d'un fait sourcé.

Le composant d'avertissement ne s'affiche *que* en mode VFX : il n'existe pas de bandeau dormant en mode données.

8.5 État d'avancement

Toutes les tâches du mode VFX n'ont pas été menées à terme pendant la durée du stage. Le tableau 8.2 distingue ce qui est livré de ce qui reste, sans embellissement.

Table 8.2. État d'avancement des tâches du mode VFX.

Tâche	État	Contenu
VFX-001	Livré	Amendement constitutionnel, spécification et plan
VFX-002	Livré	Bascule de mode, bandeau, mise en page atténuée
VFX-003	Livré	Bus d'entrée, pilote de niveau d'eau, plafonnement TRI
VFX-006	Livré	Politique de caméra, exclusion mutuelle Windy
VFX-008	Livré (partiel)	Isolation des modes, pas de double nappe d'eau
VFX-004	Restant	Dépendance Three.js et couche personnalisée Mapbox
VFX-005	Restant	Grille de terrain et nuanceur d'eau
VFX-007	Restant	Réalisateur caméra (« Play scenario »)
VFX-009	Optionnel	Post-traitement
VFX-010	Optionnel	Effets secondaires vent et feu

Un point d'honnêteté sur cet état : le niveau d'eau calculé par le mode VFX n'alimente pas encore de couche de rendu 3D, puisque la tâche VFX-004 n'est pas achevée. En revanche, ce niveau n'est pas du code dormant : il alimente l'étiquette de données partielles du bandeau, ce qui est un usage réel — la constitution interdit le code non référencé.

8.6 Bilan de la démarche

L'intérêt de ce chapitre n'est pas le rendu cinématique lui-même, mais la manière dont une tension produit a été traitée. Trois éléments ont rendu la fonctionnalité acceptable :

1. **un amendement explicite** — la règle a été changée avant le code, pas après ;
2. **une étanchéité vérifiée par des tests** — le mode VFX ne peut pas modifier le contrat ;
3. **un étiquetage obligatoire** — le spectateur sait toujours qu'il regarde une illustration.

Sans ces trois éléments, la fonctionnalité aurait été soit refusée, soit introduite en silence — et dans le second cas, elle aurait probablement érodé la confiance dans l'ensemble du produit.

CHAPITRE 9

Qualité, tests et limites connues

9.1 La stratégie de test

La suite de tests repose sur trois principes.

9.1.1 Hors-ligne par défaut

La suite tourne sans accès réseau : les sources sont remplacées par des *fixtures* capturées et des simulations. Les appels réels à Géorisques ou à la BDNB constituent un étage de tests séparé.

Cette séparation n'est pas une commodité, c'est une nécessité : une suite qui dépend de services tiers est lente, instable, et échoue pour des raisons qui n'ont rien à voir avec le code. Une suite qui échoue sans raison est une suite que l'on finit par ignorer.

9.1.2 La dégradation testée aussi rigoureusement que le nominal

C'est l'exigence FR-22 de la spécification, et probablement la plus structurante. Chaque source est éteinte indépendamment, dans chacun de ses modes de défaillance : délai d'attente, erreur 5xx, charge malformée, réponse vide. Dans tous les cas, la réponse attendue est un HTTP 200 partiel structuré, avec les aléas affectés à `present=null` et une erreur renseignée. Jamais un 500, jamais une exception non gagnée.

9.1.3 Les tests d'abord

Le workflow impose d'écrire le test avant l'implémentation : un commit `test(TXXX)` qui échoue, puis un commit `feat(TXXX)` qui passe. Deux règles complémentaires : les tests existants ne sont jamais modifiés pendant une tâche de fonctionnalité, et un test jugé erroné est corrigé dans un change set dédié.

9.2 Inventaire de la suite

Table 9.1. Les modules de test backend et leur objet.

Module	Objet
`test_post_canonical_contract.py`	Forme canonique de bout en bout
`test_diagnostic_record_schema.py`	Champs interdits, optionnels toujours présents, énumération de résolution
`test_georisques_wfs.py`	Parseur GML, ordre des axes, proximité, boîte englobante
`test_wfs_integration.py`	Client dédié, absence de contamination
`test_adresse_georisques.py`	Jointure adresse → aléas
`test_flood_alea.py`	Classes TRI, absence ≠ panne
`test_certification.py`	Suite de certification à réponse connue
`test_geocoding_hardening.py`	Arrondissements, seuil 0,4, 429
`test_api_key_auth.py`	401/403, exemptions, clés hors journaux
`test_result_cache.py`	Clé de cache, précision, expiration TTL
`test_upstream_throttle.py`	Limitation de débit amont
`test_budget_guard.py`	Compteur mensuel, rejet du lot seul
`test_chaos_degradation.py`	Matrice de pannes par source
`test_internal_batch.py`	Isolation des éléments, équivalence unitaire
`test_rate_limit.py`	Limitation entrante
`test_report_agent.py`	Rapport narratif et repli gabarit
`test_hydro.py`	Tracé hydrographique
`test_panoramax.py`	Photographie et crédit
`test_health.py`	Sondes de vie
`test_latency_report.py`	Percentiles de latence
`test_api_diagnostic_offline.py`	Pipeline hors-ligne

Côté frontend, 21 modules de test couvrent le moteur de scénarios, la boucle complète, les seuils, les vignettes, le mode VFX et l'isolation des modes.

Table 9.2. Volumétrie de la suite de tests au terme du stage.

Périmètre	Tests	Exécution
Backend (pytest, hors-ligne)	214	< 60 s
Frontend (Vitest)	143	quelques secondes
Total	*357*	

9.3 La suite de certification

La certification FR-11 mérite un développement à part. Pour quatre aléas — inondation/PPRI, mouvement de terrain, feu de forêt, avalanche — des fixtures ont été capturées pour une adresse *connue à l'intérieur* d'un périmètre et une adresse *connue à l'extérieur*, chacune vérifiée contre les cartes officielles du portail Géorisques.

L'attendu est strict :

— à l'intérieur : `present=true` et `resolution="per-building"` ;

— à l'extérieur : `present=false`.

Ce n'est pas un test unitaire de plus, c'est la **porte de signature de la version 1** : il prouve que le test géométrique dit vrai, pas seulement qu'il s'exécute. La collecte des fixtures est la partie longue de la tâche.

Un script de capture est conservé dans le dépôt, sous le nom `record_certification_fixtures.py`, avec les coordonnées candidates et l'attendu pour chaque cas — par exemple la vallée du Var à Nice comme cas exposé, et la colline de Cimiez comme cas non exposé.

Honnêteté de la donnée

Cette suite est écrite, mais sa *qualification humaine* — c'est-à-dire la confrontation visuelle des fixtures aux cartes officielles — était encore en attente à la fin du stage. Le code est là, les fixtures sont là, la validation finale par un humain reste à faire. C'est une limite, et il vaut mieux l'énoncer que la laisser découvrir.

9.4 Les portes de non-régression

Outre les tests, deux mécanismes automatiques protègent les invariants.

9.4.1 Porte de recherche sur les champs interdits

Une vérification échoue si les identifiants `niveau`, `score` ou `recommandations` apparaissent dans un schéma émis. C'est la traduction automatique de l'invariant « pas de score ».

9.4.2 Porte de recherche sur les surfaces supprimées

Après chaque suppression, une vérification confirme qu'aucune référence ne subsiste vers les modules retirés — `copernicus`, `mistral`, `lidar`, `rapport-narratif`. La règle « pas de code dormant » devient ainsi vérifiable mécaniquement.

9.5 Le test d'équivalence du lot

Un test mérite une mention : il vérifie que la sortie d'un élément de lot est *identique* à celle d'un appel unitaire pour la même adresse, à l'horodatage près.

C'est la traduction de l'invariant « le lot est N fois l'unitaire ». Sans ce test, deux implémentations peuvent diverger lentement : un champ ajouté d'un côté seulement, un arrondi différent, un ordre d'aléas qui change. Le test rend cette dérive impossible.

9.6 Les limites connues

Un rapport de stage honnête doit énumérer ce qui ne fonctionne pas ou pas encore. Le tableau 9.3 recense les limites assumées.

Table 9.3. Limites connues du système au terme du stage.

Limite	Description et statut
Cache non persistant en production	En serverless, le système de fichiers de la fonction est éphémère : le cache de rapports ne survit pas. Chaque rapport consomme un appel au modèle. Documenté.
Store de lot en mémoire	Le magasin de lots est en mémoire, mono-processus, sans persistance. Perdu au redémarrage. Suffisant pour valider le schéma, pas dimensionné pour « des centaines de milliers d'adresses ».
Authentification simulée	Sans configuration Supabase, l'authentification reste en mode démonstration : la connexion fonctionne sans comptes réels. Non utilisable en production externe.
RGA non vectoriel	Le retrait-gonflement des argiles reste communal. Un câblage sur une source vectorielle (par exemple les aléas argile du BRGM) demande un amendement constitutionnel.
Certification non qualifiée	La suite de certification à réponse connue attend sa validation humaine contre les cartes officielles.
Mode VFX incomplet	Les tâches VFX-004, VFX-005 et VFX-007 (couche Three.js, terrain, réalisateur caméra) ne sont pas livrées.
Calibration de latence	Le budget de performance (p95 $\leq$ 10 s) est proposé, non encore calibré sur une campagne de mesures réelles. Un harness de mesure existe (`latency_report.py`).
Smoke test manuel non exécuté	La vérification manuelle de bout en bout du moteur de scénarios sur une adresse certifiée TRI avec pluie prévue n'a pas été exécutée : elle demande un jeton Mapbox et des conditions météo particulières.

Honnêteté de la donnée

Le dernier point de ce tableau est le plus important à assumer. Une fonctionnalité peut être couverte par des tests automatisés et n'avoir jamais été vue fonctionner de bout en bout sur une donnée réelle. Ce n'est pas la même chose. La vérification manuelle reste due.

9.7 Performance

Le budget de performance proposé par la constitution §6 est le suivant :

— p95 $\leq$ 10 s pour `POST /diagnostic/adresse` avec toutes les sources saines ;
— le mode dégradé ne dépasse pas deux fois ce budget — un échec déclenche un repli, pas une tempête de réessais ;
— toute régression supérieure à 25 % bloque la fusion.

Ces chiffres sont présentés comme *proposés*, à calibrer par la mesure. Un point d'attention est identifié : la résolution géométrique interroge actuellement les couches WFS de façon séquentielle, chacune avec son délai d'attente et sa réattente. Si les mesures dépassent le budget, la parallélisation des requêtes de couches est prévue comme tâche conditionnelle (T017) — plutôt que de relever silencieusement le budget.

Honnêteté de la donnée

Relever un budget de performance pour qu'il corresponde à la mesure est une manière élégante de ne jamais échouer. Le projet a choisi l'inverse : le budget est un engagement, et s'il n'est pas tenu, c'est le code qui change.

CHAPITRE 10

Déploiement et mise en production

Un service non déployé n'est pas un service. Ce chapitre décrit l'architecture de déploiement retenue, les obstacles rencontrés, et la vérification qui a été faite sur l'instance en ligne.

10.1 L'objectif : une URL, zéro configuration

Le choix d'architecture de déploiement a été guidé par une contrainte d'usage : un utilisateur qui reçoit un lien doit pouvoir l'ouvrir et l'utiliser. Il ne doit pas avoir à configurer une URL de backend, ni à comprendre un problème de CORS.

Cela a conduit à un déploiement **mono-projet** : le frontend statique et le backend FastAPI servis depuis le même domaine.

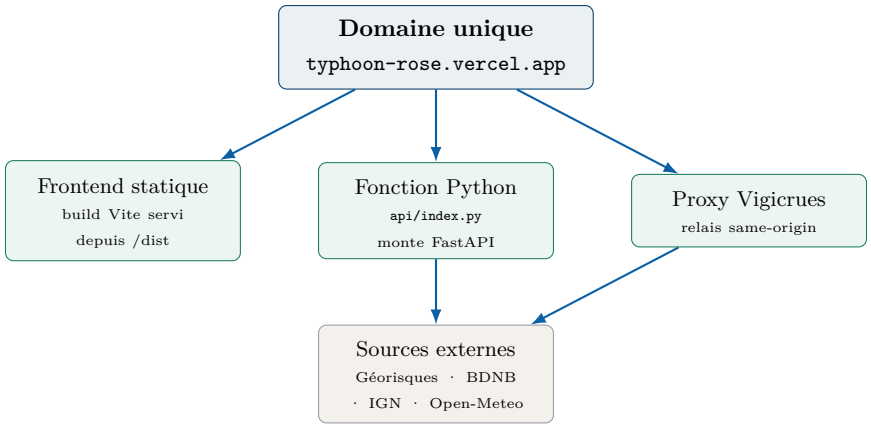


Figure 10.1. Architecture de déploiement mono-domaine. Tout passe par une seule URL : pas de CORS, pas de variable d'environnement côté client.

10.2 Le principe du mono-domaine

Le fichier de configuration à la racine du dépôt orchestre l'ensemble :

— **build** — installation et compilation du frontend, servi depuis le répertoire de sortie ;

— **rewrites** — les chemins `/api/*`, `/diagnostic/*` et `/health` sont routés vers la fonction Python qui monte l'application FastAPI ;

— **proxy Vigicrues** — un relais vers le service de vigilance crues, nécessaire car ce service n'envoie pas d'en-têtes CORS. En développement, c'est le proxy de Vite qui joue ce rôle ; en production, c'est une règle de réécriture ;

— **fallback SPA** — toute route cliente inconnue est servie par l'application, pour que `/zone` ou `/dashboard` ne renvoient pas une erreur 404.

Côté client, une règle importante : le frontend appelle le backend **en même origine** (URLs relatives) dès que la page n'est pas servie depuis un hôte local. Aucune URL n'est à configurer, aucun CORS n'est à gérer.

Invariant produit

Un `VITE_API_BASE` qui pointe vers `http://127.0.0.1:8000` est **ignoré en production** : il désignerait la machine du visiteur, ce qui produirait un message trompeur du type « backend inaccessible ? ». La résolution d'URL ignore un override local sur un hôte déployé.

10.3 Les obstacles rencontrés

Le déploiement a été la partie la plus itérative du stage. L'historique des commits en garde la trace, et les enseignements valent d'être consignés.

Table 10.1. Obstacles de déploiement et résolutions.

Symptôme	Cause et correction
« Could not find a top-level `app` »	La fonction Python doit exposer le nom `app` **au niveau racine du module**, hors bloc conditionnel. Un diagnostic de démarrage peut rester, mais l'affectation doit remonter sans indentation.
« pattern `api/index.py` doesn't match any Serverless Functions »	Message trompeur : en configuration moderne, la fonction est écartée silencieusement si le nom racine est absent. Le message ne mentionne pas la vraie cause.
`FUNCTION_INVOCATION_FAILED` (500)	Variables d'environnement présentes mais vides. La configuration a été durcie pour ignorer les variables vides plutôt que de les traiter comme définies.
Le répertoire statique monté au mauvais endroit	Une configuration héritée montait le résultat du build sous un sous-chemin. Correction par une configuration moderne.
« Vigilance crues indisponible »	Sans règle de proxy, `/vigicrues/...` tombait dans le fallback SPA et renvoyait du HTML au lieu de données.
Routes clientes en 404	Ajout du fallback SPA.

Une leçon se dégage de cet ensemble : **les messages d'erreur d'une plateforme de déploiement désignent souvent le symptôme, pas la cause**. Le message « le motif ne correspond à aucune fonction » n'indique pas que le motif est faux, mais que la fonction a été écartée pour une autre raison. Il a fallu remonter à la cause racine — la visibilité du nom racine — pour corriger.

10.4 Comportements propres au serverless

Le passage à une fonction serverless introduit des différences qu'il faut connaître avant une démonstration :

— **Démarrage à froid** — le réveil d'une fonction froide prend de 5 à 10 s, et un diagnostic complet peut atteindre environ 15 s à froid. C'est normal sur l'offre gratuite ; l'astuce opérationnelle consiste à appeler la sonde de vie une fois avant une démonstration.

— **Cache non persistant** — le système de fichiers de la fonction est éphémère. Chaque rapport narratif consomme donc un appel au modèle, et la prose varie légèrement entre deux requêtes identiques.

— **Clé Windy liée au domaine** — l'API météo n'autorise une clé que sur les domaines déclarés. Le domaine de production doit y figurer, sinon les couches météo échouent. La console nomme alors explicitement le domaine à ajouter — un bon exemple de message d'erreur actionnable.

10.5 Vérification de l'instance en ligne

Une instance de référence a été déployée et vérifiée le 15 septembre 2026. Le tableau 10.2 reproduit les contrôles effectués et leurs résultats.

Table 10.2. Contrôles de l'instance déployée, vérifiés le 15 septembre 2026.

Contrôle	Résultat
`/` et `/zone`	200 — application servie (fallback SPA)
`/health`, `/health/detailed`	200, `{"status":"ok"}`
`/api/flood-alea` (Quai de la Rapée)	200 — `in_tri: true`, classes TRI réelles
`/api/meteo`, `/api/hydro`, `/api/photo`	200
`/api/geocode/search`	200 — géocodage réel
`POST /diagnostic/adresse` (Quai de la Rapée)	200 en environ 9 à 16 s
`POST /api/report`	200, repli non utilisé, prose française
`POST /api/report/stream`	200 en environ 1,6 s, avec les événements `header`, `summary`, `mitigations`, `confidence`, `appendix`, `patch`, `done`
Jeton Mapbox injecté dans le bundle	oui

10.6 Adresses de référence

Pour que la démonstration soit reproductible, trois adresses de référence ont été identifiées et vérifiées.

Table 10.3. Adresses de référence pour la démonstration.

Adresse	Rôle	Résultat vérifié
Quai de la Rapée, 75012 Paris	Démonstration principale, eau sur le bâtiment	`available: true`, Moyen 0–1 m, Faible 2–3 m, bassin Seine
10 Quai de la Charente, 75019 Paris	Absence honnête : dans un TRI sans classe au point	`in_tri: true`, 6/13 aléas, fiche BDNB 699 m^2
Adresse intérieure hors TRI (ex. Brou, 28300)	Contraste « hors TRI ≠ jamais inondé »	`available: false`, `in_tri: false`

Le parcours de démonstration suggéré exploite délibérément le contraste : la première adresse montre le produit en fonctionnement nominal, la deuxième montre la rigueur — un message distinct, aucun chiffre inventé — et la troisième montre l'honnêteté sur l'absence.

10.7 Alternative de déploiement

Une seconde option de déploiement est documentée pour le cas où les fonctions serverless seraient trop lentes : backend sur une plateforme d'hébergement classique (blueprint prêt dans le dépôt) et frontend statique. Le compromis est inverse : démarrage plus lent après inactivité (environ 30 s de réveil sur l'offre gratuite), mais performances d'exécution plus régulières. Cette option existe parce qu'un choix d'infrastructure est un arbitrage, pas une évidence.

CHAPITRE 11

Résultats et validation

11.1 Le livrable

Au terme du stage, TYPHOON est un service déployé et vérifié en ligne, composé de :

— un backend FastAPI de 70 fichiers Python, 8 routes fonctionnelles et 10 connecteurs ;
— un frontend React/TypeScript de 68 modules, avec une carte Mapbox 3D, un moteur de scénarios et un mode de rendu cinématique ;
— **357 tests automatisés** hors-ligne (214 backend, 143 frontend) ;
— un contrat de données versionné, dont la conformité est imposée par le schéma et vérifiée par des tests ;
— une constitution d'ingénierie de 8 sections et un amendement ;
— une documentation de travail complète : brainstorm, concept, spécification (28 exigences), plan (18 tâches), plan de scénarios (10 tâches), spécification et plan VFX.

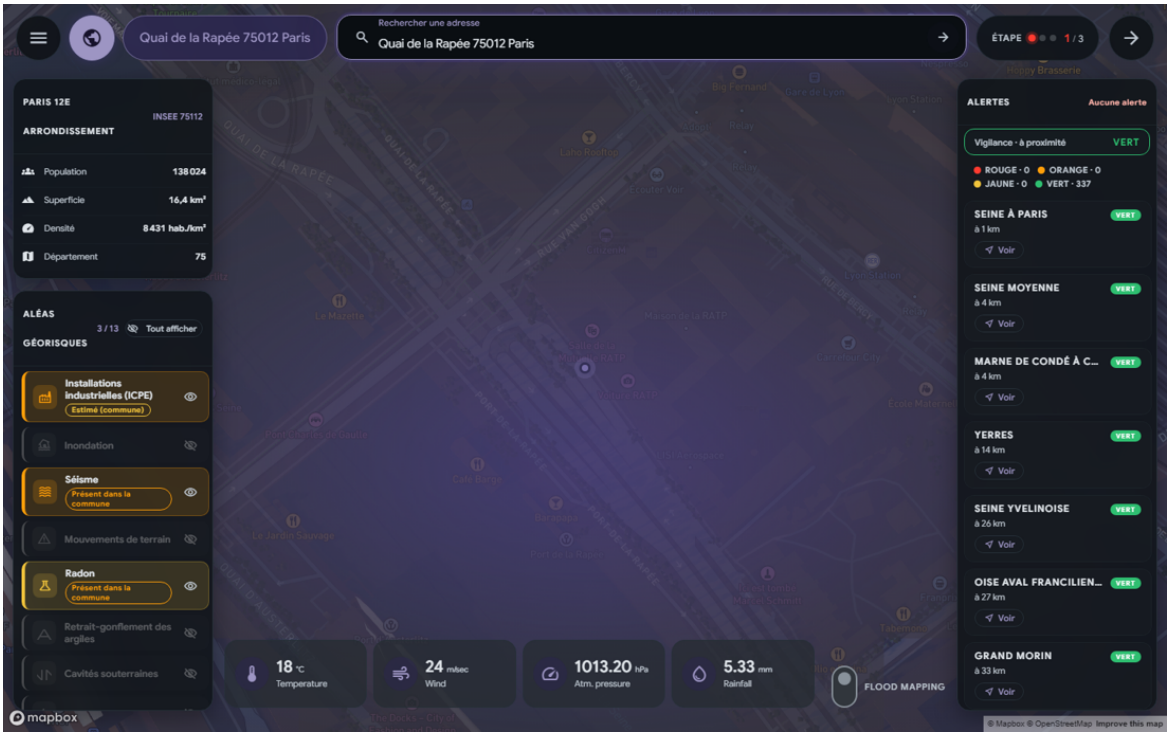


Figure 11.1. Étape 1 — diagnostic. Les 13 aléas Géorisques avec leur badge de résolution, la fiche BDNB du bâtiment, et une photographie terrain Panoramax orientée sur le point, sur une carte 3D.

11.2 Le contrat en pratique

Le contrat produit pour l'adresse de démonstration principale illustre la jointure. On y lit, pour chaque aléa, trois informations indissociables : le fait, sa source, et sa résolution.

Table 11.1. Lecture du contrat pour l'adresse de démonstration principale.

Aléa	`present`	Résolution	Origine du fait
Inondation	`true`	`per-building`	Test point-in-polygon réussi contre le périmètre PPR inondation
Sismicité	selon zone	`commune-level`	Zonage sismique décrétal — résolution maximale existante
Radon	selon commune	`commune-level`	Classement de potentiel communal
RGA	selon commune	`commune-level-estimate`	Estimation communale assumée, source vectorielle non câblée
Feu de forêt	`true/false`	`per-building`	Test géométrique contre le périmètre

Le point important n'est pas la valeur de chaque ligne, mais le fait que la troisième colonne existe. Un lecteur du contrat sait *toujours* à quel niveau l'information a été établie, et peut donc décider du crédit à lui accorder.

11.3 Ce que le produit démontre

11.3.1 La jointure existe et elle est vérifiée

Le résultat central est que la jointure entre exposition et vulnérabilité, à l'échelle du bâtiment, est établie par un test géométrique réel et non par une approximation communale. Pour les aléas couverts par un périmètre vectoriel, le fait porte effectivement sur le bâtiment.

11.3.2 Le refus du score est tenable

La suppression du score composite a été testée jusqu'au bout : le schéma refuse la sérialisation de champs de score, une porte de recherche échoue si un tel champ apparaît, et la valeur produit se reconstruit sur d'autres fondations — la provenance et la résolution. Le produit est démontrable sans son chiffre.

11.3.3 Les modes de défaillance sont explicites

La trichotomie `true/false/null` et la matrice d'erreurs font que le produit ne confond jamais « pas de risque » et « je ne sais pas ». C'est peut-être la propriété la plus difficile à démontrer sur un produit de données, et elle est ici vérifiée par une suite de tests de dégradation.

11.3.4 Le moteur de scénarios relie la donnée au jugement

La carte ne montre pas une donnée : elle montre la jointure. Et le moteur de scénarios transforme une classe de retour abstraite (« 100 ans ») en une montée des eaux lisible, sans jamais inventer un niveau.

11.4 Chiffres clés

Table 11.2. Synthèse quantitative du projet.

Indicateur	Valeur	Commentaire
Commits	76	du 14 août au 15 septembre 2026
Fichiers Python (backend)	70	dont 10 connecteurs, 8 routes, 7 services
Modules TypeScript (frontend)	68	dont 20 dans le moteur de scénarios
Modules de test	45	24 backend, 21 frontend
Tests automatisés	357	214 backend, 143 frontend, hors-ligne
Exigences fonctionnelles	28	FR-01 à FR-28, chacune avec critères d'acceptation
Tâches planifiées	18 + 10	plan principal + plan de scénarios
Aléas normalisés	13	référentiel Géorisques complet
Aléas certifiés	4	inondation, mouvement de terrain, feu, avalanche
Sources externes intégrées	9	dont 7 publiques
Sections constitutionnelles	8	dont un amendement (§2.1, mode VFX)
Décisions documentées	13 + 5	cadrage + moteur de scénarios

11.5 Validation

La validation du travail repose sur quatre niveaux complémentaires.

Tests automatisés.
357 tests hors-ligne, exécutés en moins d'une minute pour le backend. Ils couvrent le chemin nominal, la dégradation, les invariants de schéma et l'équivalence unitaire/lot.

Vérification en ligne.
Une instance déployée, contrôlée endpoint par endpoint le 15 septembre 2026 (tableau 10.2).

Confrontation au métier.
Les décisions structurantes — refus du score, variables brutes plutôt qu'un indice, livraison à la fois interactive et en lot — proviennent d'échanges réels avec un actuaire et un professionnel du secteur, pas d'une hypothèse de conception.

Reproductibilité de la démonstration.
Trois adresses de référence documentées, avec le résultat attendu pour chacune, permettent de rejouer la démonstration à l'identique.

11.6 Apports personnels

Au-delà du livrable, ce stage a produit plusieurs apprentissages que je considère transférables.

1. **La valeur d'un produit de données tient à sa traçabilité, pas à son volume.** Une jointure modeste mais vérifiable vaut mieux qu'une agrégation large et approximative.
2. **Le refus est une décision d'ingénierie.** Supprimer le score, supprimer quatre connecteurs, supprimer des milliers de lignes a *augmenté* la valeur du livrable.
3. **Une panne silencieuse est plus dangereuse qu'une erreur visible.** Le bug de la boîte englobante répondait correctement avec une information fausse. C'est ce qui m'a conduit à traiter la trichotomie `true`/`false`/`null` comme une exigence de premier rang.

4. **Un correctif non documenté sera défait.** Travailler avec des agents de programmation rend cette règle vitale : l'invariant doit exister dans un document lu avant le code.
5. **Les sources publiques ont des pièges non documentés.** Le client HTTP partagé, la boîte englobante, l'ordre des axes GML : rien de tout cela n'est dans la documentation officielle. Le savoir s'accumule par la mesure et doit être écrit.

11.7 Impact potentiel

Il serait malhonnête de présenter TYPHOON comme un produit prêt à être vendu. C'est un prototype fonctionnel qui démontre une thèse : la jointure aléa × vulnérabilité par bâtiment est réalisable à partir de sources publiques, avec une traçabilité compatible avec les exigences d'un audit.

Le chemin vers un usage réel passe par des étapes identifiées : qualification humaine de la certification, calibration de latence, authentification réelle, persistance du store de lot, et surtout validation de l'intérêt par un pilote chez un assureur. La valeur du prototype est d'avoir rendu cette discussion concrète — on ne débat plus de l'idée, on débat d'une URL qu'on peut ouvrir.

CHAPITRE 12

Bilan critique

Un rapport de stage qui ne présente que des réussites n'est pas un rapport : c'est une plaquette. Ce chapitre expose ce qui a été difficile, ce qui a mal tourné, et ce que je ferais différemment.

12.1 Ce qui a bien fonctionné

12.1.1 La remise en question du cadrage

Le moment le plus productif du stage a été la session de recentrage du 25 août. Elle a consisté à admettre qu'un travail déjà réalisé — quatre connecteurs, une couche de notation, un tableau de bord — n'apportait pas la valeur attendue, et à le supprimer.

Cette décision a été facilitée par un critère unique et brutal : *l'acheteur pourrait-il obtenir ceci directement depuis la source ?* Un critère unique est plus facile à appliquer qu'une liste de priorités, parce qu'il ne laisse pas de place à la négociation.

12.1.2 La confrontation au terrain métier

Les échanges avec un actuaire et un professionnel de l'assurance ont directement réécrit la direction produit. Le refus du score composite, la préférence pour les variables brutes et l'exigence de provenance viennent de ces conversations, pas d'une intuition.

La leçon est générale : **il est moins coûteux de se tromper auprès d'un utilisateur que de se tromper tout seul pendant deux mois**. Ces échanges ont coûté quelques heures et ont évité de construire un produit que personne n'aurait voulu.

12.1.3 La discipline documentaire

Le triptyque constitution / spécification / plan a permis de stabiliser un projet dont une part importante du code a été écrite en collaboration avec des agents de programmation. Sans ce cadre, le projet aurait très probablement dérivé : chaque session d'agent aurait pu défaire silencieusement une décision antérieure.

12.2 Ce qui a mal fonctionné

12.2.1 Les bugs sémantiques répétés

Trois fois de suite, un comportement *fonctionnel* a dû être corrigé parce qu'il était *faux* :

1. les clés de scénario ne correspondaient pas ;
2. un seuil unique confondait dommages et affichage ;
3. le niveau d'eau dépendait de la météo du jour.

Dans les trois cas, le code s'exécutait sans erreur. Le fait que ces bugs soient apparus *après* la mise en service montre une faiblesse de méthode : les tests vérifiaient que le code ne plantait pas, pas qu'il disait vrai. Une partie de la réponse a été la suite de tests de scénarios

(`scenarioLoop.test.ts`), qui verrouille la chaîne complète de la clé jusqu'à l'eau dessinée. Mais cette suite est arrivée *après* les bugs.

Ce que je retiens : **pour un produit de données, un test qui vérifie l'absence d'exception est presque inutile**. Ce qui compte est un test qui vérifie une valeur attendue sur une entrée connue.

12.2.2 La dérive initiale

La phase 1 a produit du code qui a dû être supprimé. Avec le recul, le problème n'était pas le travail lui-même — explorer quatre sources était nécessaire pour comprendre ce qui avait de la valeur — mais l'*absence de critère explicite* pendant cette exploration. Il a fallu construire le critère d'abord, puis mesurer le travail à l'aune de ce critère.

Si c'était à refaire, j'écrirais la question « l'acheteur pourrait-il obtenir ceci depuis la source ? » dès la première semaine, avant d'écrire la première ligne de connecteur.

12.2.3 L'instabilité du déploiement

Le déploiement a consommé une part disproportionnée du temps. Les messages d'erreur de la plateforme désignaient systématiquement le symptôme, jamais la cause. Le message « le motif ne correspond à aucune fonction » a coûté plusieurs itérations alors que la cause réelle — la visibilité du nom racine dans le module — n'était mentionnée nulle part.

La leçon : **sur une plateforme d'hébergement, il faut diagnostiquer par l'expérience contrôlée, pas par le message d'erreur**. Réduire la surface, changer une variable à la fois, observer.

12.2.4 Le périmètre du mode VFX

Le mode VFX a été spécifié, encadré, partiellement implémenté — et non terminé. Trois tâches sur dix restent ouvertes. C'est un périmètre trop ambitieux pour la durée disponible, et la conséquence est un demi-livrable.

Le point positif est que la partie livrée est la partie *gouvernance* : l'amendement, l'étiquetage, le bus d'entrée et les garanties. C'est la partie qui protège le produit. Le rendu 3D lui-même est la partie amusante, et c'est celle qui manque.

12.3 Ce que je ferais différemment

Table 12.1. Choix passés et alternatives envisagées.

Ce qui a été fait	Ce que je ferais autrement
Explorer quatre sources avant de définir le critère de valeur	Écrire le critère de valeur dès le premier jour, puis explorer
Écrire les tests de scénarios après la mise en service	Écrire un test de valeur attendue (clé → pic → profondeur) dès l'implémentation du moteur
Traiter le déploiement au fil de l'eau	Déployer une version minimale dès la première semaine, pour découvrir l'infrastructure tôt plutôt que tard
Spécifier dix tâches VFX	Spécifier trois tâches VFX (bascule, étiquetage, garanties) et garder le rendu pour une phase ultérieure
Garder la calibration de latence pour la fin	Mesurer dès que le pipeline est stable, pour que le budget soit un engagement tenu et non une hypothèse

12.4 Compétences acquises

Table 12.2. Compétences mobilisées et développées.

Domaine	Acquis
Systèmes d'information géographique	WFS et GML 3.2, test point-in-polygon par lancer de rayons, proximité géométrique, projections et ordre des axes, boîtes englobantes
Ingénierie backend	FastAPI, Pydantic v2, asynchrone avec `httpx`, conception de contrat de données, gestion de cache et de quotas
Ingénierie frontend	React et TypeScript, Mapbox GL JS (extrusion, couches personnalisées, terrain), animation synchronisée à un état applicatif
Qualité logicielle	Tests hors-ligne avec fixtures, tests de dégradation, tests de propriété, portes de recherche CI
Méthodologie	Développement piloté par les spécifications, gouvernance par document, collaboration structurée avec des agents de programmation
Déploiement	Architecture serverless mono-domaine, réécritures et proxies, diagnostic d'incidents de plateforme
Domaine métier	Souscription, Solvabilité II, régime CatNat, non-stationnarité du risque climatique, distinction entre donnée et jugement

12.5 Appréciation personnelle

Ce stage m'a surtout appris qu'un projet technique se joue moins sur la difficulté du code que sur la justesse des décisions. Écrire un parseur GML est un exercice ; décider qu'il ne faut *pas* produire de score est une décision. La seconde a demandé plus de courage et a eu plus d'impact.

J'en retiens une posture : chercher la formulation la plus honnête possible du problème, et s'y tenir même quand elle impose de supprimer du travail. Le recentrage du 25 août a été inconfortable sur le moment — il fallait admettre que plusieurs semaines de production ne servaient pas le produit

— mais c'est ce qui a donné au livrable sa cohérence.

Le projet n'est pas terminé. Il est, en revanche, défendable : chaque choix est documenté, chaque fait porte sa source, et chaque limite connue est écrite plutôt que cachée. C'est probablement la définition d'un travail honnête.

CHAPITRE 13

Conclusion et perspectives

13.1 Rappel de la problématique

Ce stage partait d'un constat simple : les données publiques de risque climatique existent, mais la jointure entre l'exposition d'un bâtiment et sa vulnérabilité n'existe nulle part. Un souscripteur qui doit évaluer un dossier le fait aujourd'hui à l'œil, en croisant deux portails, sans traçabilité.

La question posée était donc : peut-on construire, à partir de sources publiques uniquement, un service qui réponde à « ce bâtiment est-il exposé, et qu'est-ce qui le rend vulnérable ? », avec une qualité d'information suffisante pour qu'un professionnel de l'assurance s'appuie dessus ?

13.2 Réponse apportée

La réponse est oui, avec des conditions. Le stage a produit un service fonctionnel, déployé et vérifié, qui :

1. résout une adresse française et interroge les sources en parallèle ;
2. teste le point résolu contre les périmètres d'aléas par géométrie réelle, pas par approximation communale ;
3. associe à chaque aléa la description verbatim du bâtiment issue de la BDNB ;
4. porte, sur chaque fait, sa source, sa date et son niveau de résolution ;
5. refuse explicitement de produire un score ou une recommandation ;
6. distingue trois états — présent, absent, inconnu — de sorte qu'une panne ne se lise jamais comme une absence de risque ;
7. met le résultat en situation sur une carte, avec un moteur de scénarios fondé sur les classes réglementaires, sans jamais inventer un niveau ;
8. est couvert par 357 tests automatisés hors-ligne.

Les conditions sont aussi importantes que la réponse. Le service n'est pas commercialisable en l'état : l'authentification est simulée, le magasin de lots est en mémoire, la certification attend sa qualification humaine, et le budget de latence n'est pas encore calibré. Le prototype démontre une thèse ; il ne remplace pas un produit.

13.3 Les trois idées à retenir

13.3.1 Le refus du score est une position tenable

L'intuition initiale — produire un score global de risque — a été délibérément abandonnée. Ce n'est pas une concession, c'est un positionnement : sous Solvabilité II, un score tiers engage son producteur, se fait reconstruire ou refuser par l'utilisateur, et mélange des aléas qui ne se compensent pas.

Le produit a été reconstruit sur d'autres fondations — provenance et résolution — et s'est révélé démontrable sans son chiffre. Le champ de résolution, qui n'était qu'un attribut parmi d'autres, est devenu le signal de qualité central.

13.3.2 L'honnêteté sur l'incertitude est une fonctionnalité

La trichotomie `true`/`false`/`null`, la distinction entre absence et panne, l'étiquetage du mode cinématique : tout cela relève du même principe. Un produit de données ne se juge pas seulement sur ce qu'il affirme, mais sur la manière dont il dit « je ne sais pas ».

C'est aussi ce qui rend le produit utilisable en contexte réglementaire : une décision peut être justifiée si l'on sait quel fait a été établi à quel niveau.

13.3.3 La soustraction est un travail d'ingénierie

La phase la plus productive du stage a été celle où le périmètre a été réduit. Supprimer quatre connecteurs, une couche de notation, un tableau de bord et des milliers de lignes a augmenté la valeur du livrable. La règle « pas de code dormant » et l'historique Git comme unique archive ont rendu cette soustraction possible sans perte d'information.

13.4 Perspectives

Le travail identifié pour la suite se répartit en trois horizons.

13.4.1 Court terme — finir et qualifier

— **Qualifier la certification.** Confronter les fixtures des quatre aléas certifiés aux cartes officielles et signer la version 1.

— **Calibrer la latence.** Exécuter la campagne de mesures sur au moins 100 appels réels et décider, sur données, si la parallélisation des couches WFS est nécessaire.

— **Exécuter le test manuel de bout en bout.** Diagnostiquer une adresse certifiée TRI un jour de pluie et valider la boucle complète.

— **Terminer le mode VFX** — ou acter qu'il reste un démonstrateur partiel, ce qui est défendable puisque la partie gouvernance est livrée.

13.4.2 Moyen terme — rendre le service exploitable

— **Authentification réelle** — remplacer le mode démonstration par un système de comptes, condition de tout usage externe.

— **Persistance du store de lot** — remplacer le magasin en mémoire par une file de traitement, seule façon de tenir le volume.

— **Cache partagé** — sortir le cache du processus pour qu'il survive au serverless.

— **Source vectorielle pour le RGA** — câbler une source polygonale pour le retrait-gonflement des argiles, ce qui exige un amendement constitutionnel, et supprimerait la dernière étiquette `commune-level-estimate` structurelle.

13.4.3 Long terme — le produit complet

— **La trajectoire climatique.** C'est l'argument de fond du produit : la non-stationnarité du risque. Le prototype en pose la structure — horizons explicites, scénarios étiquetés, unités réelles — mais il reste à industrialiser la lecture des projections et à exposer les variables par aléa et par horizon dans le contrat.

— **La livraison en lot pour l'actuaire.** Le second acheteur attend de la donnée structurée à l'échelle d'un portefeuille, avec les mêmes sémantiques que le produit interactif. L'invariant « le lot est N fois l'unitaire » a été posé précisément pour rendre cette extension non coûteuse.

— **La certification étendue.** Passer de quatre aléas certifiés à l'ensemble des aléas couverts par un périmètre vectoriel.

13.5 Mot de la fin

Typhoon est un prototype, et il faut le présenter comme tel. Mais c'est un prototype qui démontre quelque chose de précis : *la donnée publique française permet de construire une information de risque climatique par bâtiment, traçable et défendable* — à condition d'accepter de ne pas tout savoir et de le dire.

Le stage m'a laissé une conviction : dans un domaine où la donnée est abondante, la valeur ne se crée pas en ajoutant des sources ou des indicateurs, mais en établissant avec rigueur une jointure que personne n'avait faite, et en assumant explicitement les limites de ce que l'on affirme.

> *Un produit de données honnête ne dit pas seulement ce qu'il sait.*
> *Il dit aussi comment il le sait, et à quel point il le sait.*

ANNEXE A

Référence du contrat canonique

Cette annexe reprend la spécification complète du contrat de données, telle qu'elle figure dans la spécification du projet. Elle est destinée à un intégrateur qui souhaite consommer l'API.

A.1 Structure complète

```
POST /diagnostic/adresse
body: { "adresse": "10 rue de Rivoli, 75004 Paris" }

200 ->
{
  "schema_version": "1.0",
  "adresse": {
    "saisie": "...",              // entree brute
    "normalisee": "...",          // libelle du geocodeur
    "citycode": "75056",
    "postcode": "75004",
    "city": "Paris",
    "lat": 48.8566,
    "lon": 2.3522,
    "geocode_score": 0.95
  },
  "aleas": [                      // 13 objets, referentiel complet
    {
      "code": "inondation",
      "libelle": "Inondation",
      "present": true,            // true | false | null
      "present_commune": true,
      "zonage": "Dans un perimetre PPR inondation",   // texte verbatim
      "hauteur_eau_m": 1.5,       // optionnels specifiques a l'alea,
      "zone_sismique": null,      // TOUJOURS presents, null si non applicable
      "catnat_historique": [],
      "per_building": {           // present uniquement si teste geometriquement
        "method": "point-in-polygon",  // "point-in-polygon" | "proximity"
        "radius_m": null,              // echo de la proximite si applicable
        "count": 2
      },
      "source": "georisques",
      "url_detail": "https://www.georisques.gouv.fr/risques/inondations",
      "erreur": null,             // string | null
      "resolution": "per-building"
      // "per-building" | "commune-level" | "commune-level-estimate"
    }
  ],
  "bdnb": {                       // bloc entier null si BDNB indisponible
    "donnees": {
      // ~139 champs BDNB verbatim
      // batiment = batiment principal
      // autres_batiments_meme_adresse = [...]
    },
    "_source": {
      "provider": "BDNB",
      "url": "https://api.bdnb.io",
```

```
      "attribution": "Source BDNB (CSTB) - donnees a jour au <millesime>",
      "recuperee_le": "2026-08-25T10:00:00+00:00"
    }
  },
  "georisques_source": {
    "provider": "Georisques",
    "url": "https://www.georisques.gouv.fr",
    "attribution": "Source Georisques (BRGM / MTE) - donnees a jour au <millesime>",
    "recuperee_le": "2026-08-25T10:00:00+00:00"
  },
  "erreurs_partielles": [],
  "genere_le": "2026-08-25T10:00:01+00:00",
  "avertissement": "Ce rapport agrege les donnees publiques Georisques..."
}
```

Listing A.1. Contrat canonique `DiagnosticRecord` v1.0.

A.2 Champs interdits

Les champs suivants ne peuvent pas exister dans le contrat. Leur présence fait échouer la validation du schéma, et une porte de recherche automatique échoue si l'un d'eux apparaît dans un schéma émis :

— `niveau` et les bandes D03 ;
— tout score numérique, agrégé ou composite ;
— `recommandations` et toute évaluation générée par IA ;
— `copernicus` dans le contrat canonique v1.

A.3 Règles de sérialisation

Table A.1. Règles de sérialisation à respecter par tout consommateur.

Règle	Énoncé
Optionnels toujours présents	Les champs spécifiques à un aléa sont toujours présents, à `null` lorsqu'ils ne s'appliquent pas. Une clé n'est jamais absente.
Exception `per_building`	L'objet `per_building` n'existe que si le point a été testé géométriquement. Sa présence porte l'information.
BDNB verbatim	Les clés BDNB reçues et émises sont identiques — vérifié par test de conformité.
Attribution non vide	Chaque bloc `_source` porte une chaîne d'attribution non vide.
Résolution contrainte	Le RGA ne peut sérialiser aucune autre valeur que `commune-level-estimate`.

A.4 Le référentiel des treize aléas

Table A.2. Les treize aléas, leur libellé et leur niveau de résolution attendu.

Code	Libellé	Résolution attendue
`icpe`	Installations classées	`per-building` ou `commune-level`
`inondation`	Inondation	`per-building`
`sismicite`	Sismicité	`commune-level` (décret, permanent)
`mouvement_terrain`	Mouvement de terrain	`per-building`
`radon`	Radon	`commune-level` (permanent)
`rga`	Retrait-gonflement des argiles	`commune-level-estimate` (forcé)
`cavite`	Cavités souterraines	`per-building` ou communal
`feu_foret`	Feu de forêt	`per-building`
`avalanche`	Avalanche	`per-building`
`canalisations`	Canalisations (TMD)	`per-building` par proximité (200 m)
`vent_cyclonique`	Vent cyclonique	`commune-level`
`ppr`	PPR (agrégat)	`per-building`
`ssp`	Sites et sols pollués	`per-building` par proximité (200 m)

ANNEXE B

Surface de l'API et mise en œuvre

B.1 Points d'entrée

Table B.1. Points d'entrée de l'API, avec authentification requise.

Méthode et chemin	Auth	Description
`POST /diagnostic/adresse`	oui	Transaction canonique, adresse → enregistrement complet
`POST /diagnostic/batch`	oui	Soumission d'un lot (≤ 10 000 adresses)
`GET /diagnostic/batch/{id}`	oui	État et résultats du lot
`GET /diagnostic/adresse/rapport-pdf`	oui	Relais vers le PDF officiel Géorisques
`GET /diagnostic/zone/building`	oui	Fiche BDNB au point
`GET /diagnostic/zone/buildings`	oui	Fiches BDNB autour du point
`GET /api/flood-alea`	oui	Classes TRI au point
`GET /api/hydro`	oui	Tracé hydrographique
`GET /api/hydro/extend`	oui	Extension du tracé
`GET /api/meteo`	oui	Pluie horaire et débit
`GET /api/photo`	oui	Photographie Panoramax et crédit
`GET /api/report`	oui	Rapport narratif
`GET /api/report/stream`	oui	Rapport narratif en flux
`GET /api/geocode/search`	oui	Autocomplétion d'adresse
`GET /health`	non	Sonde de vie
`GET /health/detailed`	non	Sonde détaillée

B.2 Format des erreurs

Table B.2. Codes d'erreur et signification.

Code	Identifiant	Signification
401	—	Clé d'API absente
403	—	Clé d'API invalide
422	`adresse_non_trouvee`	Adresse introuvable par le géocodeur
422	`adresse_ambigue`	Score de géocodage inférieur à 0,4
429	`budget_epuise`	Budget mensuel amont épuisé (lot uniquement)
502	`geocodage_indisponible`	Géocodeur injoignable ou saturé
502	—	Exception inattendue (jamais de 500 HTML)

B.3 Commandes d'installation

```
# Backend (Python 3.11+)
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend (Node 18+)
```

```
cd frontend
npm install
npm run dev        # http://localhost:5173
```

Listing B.1. Installation et exécution en local.

Le fichier d'exemple d'environnement, à copier à la racine du dépôt, documente chaque variable nécessaire : jeton Mapbox, URL et clé Supabase, clé Windy, clé du modèle de rédaction, identifiants Copernicus, et clés d'API partenaires.

B.4 Commandes de vérification

```
# Backend (depuis backend/)
python -m pytest tests/ -q     # 214 tests hors-ligne, < 60 s
ruff check app tests           # lint

# Frontend (depuis frontend/)
npm test                       # 143 tests Vitest
npm run build                  # tsc -b && vite build
```

Listing B.2. Exécution des suites de test.

B.5 Vérification de l'instance déployée

```
BASE=https://<projet>.vercel.app
curl $BASE/health
curl "$BASE/api/flood-alea?lat=48.848&lon=2.370" | head -c 200
curl -X POST "$BASE/diagnostic/adresse" \
  -H 'Content-Type: application/json' \
  -d '{"adresse":"Quai de la Rapee, 75012 Paris"}' | head -c 200
```

Listing B.3. Contrôles de bon fonctionnement de l'instance en ligne.

Le réveil d'une fonction froide prend de 5 à 10 s, et un diagnostic complet peut atteindre environ 15 s à froid. C'est normal sur l'offre gratuite : appeler la sonde de vie une fois avant une démonstration suffit à préchauffer l'instance.

B.6 Reproduction des captures d'écran

```
cd frontend && node scripts/demo-shots.mjs
# necessite le backend et le serveur vite en local
```

Listing B.4. Génération des captures d'écran.

ANNEXE C

Glossaire

C.1 Termes métier

Souscription (*underwriting*).
Processus par lequel un assureur accepte, met à l'étude ou refuse de couvrir un risque, et détermine les conditions de la couverture.

Souscripteur.
Professionnel qui traite les dossiers de souscription au quotidien.

Actuaire.
Professionnel qui construit les modèles de tarification et de provisionnement, en s'appuyant sur des données plutôt que sur une interface.

Solvabilité II.
Cadre réglementaire européen de solvabilité des assureurs. Il impose notamment la traçabilité et la justification des décisions.

Régime CatNat.
Régime français d'indemnisation des catastrophes naturelles, fondé sur la reconnaissance de l'état de catastrophe naturelle par arrêté interministériel.

Non-stationnarité.
Propriété d'un risque dont la distribution change dans le temps. Le risque climatique est non stationnaire : le passé ne prédit pas directement le futur.

PPR.
Plan de Prévention des Risques. Document réglementaire qui délimite des zones exposées à un aléa et fixe des prescriptions.

TRI.
Territoire à Risque Important d'inondation, issu de la Directive Inondation. Classe l'exposition en quatre niveaux de période de retour.

Période de retour.
Durée moyenne entre deux occurrences d'un événement d'intensité donnée. Une crue centennale a une probabilité d'environ 1 % par an.

RGA.
Retrait-gonflement des argiles. Phénomène de déformation des sols argileux selon leur humidité, cause majeure de fissuration des bâtiments.

CatNat historique.
Liste des arrêtés de catastrophe naturelle pris pour une commune.

C.2 Termes techniques

WFS.
Web Feature Service. Standard de l'Open Geospatial Consortium pour l'échange de données géographiques vectorielles.

GML.
Geography Markup Language. Format XML d'encodage de géométries, utilisé par les services WFS. Le projet traite GML 3.2.

Point-in-polygon.
Test déterminant si un point est à l'intérieur d'un polygone. Implémenté par lancer de rayon.

Lancer de rayon.
Algorithme consistant à compter les intersections d'un rayon partant du point avec les arêtes du polygone. Un nombre impair signifie que le point est à l'intérieur.

Boîte englobante (BBOX).
Rectangle délimitant la zone d'une requête spatiale.

Proximité géométrique.
Test applicable aux géométries non polygonales (points, lignes) : le point est-il à moins d'un rayon donné ?

Ordre des axes.
Convention de notation des coordonnées (latitude/longitude ou longitude/latitude). Une erreur d'ordre fait échouer silencieusement tout test géométrique.

Verbatim.
Passage d'une donnée sans transformation : ni renommage, ni filtrage, ni recalcul.

Provenance.
Métadonnée décrivant l'origine d'un fait : fournisseur, URL, chaîne d'attribution, horodatage de collecte.

Résolution.
Niveau auquel un fait a été établi : par bâtiment, par commune, ou estimé par commune.

Mode dégradé.
Comportement du service lorsqu'une source échoue : la réponse reste valide mais partielle, avec les faits concernés marqués comme inconnus.

Fixture.
Donnée de test figée, capturée une fois depuis une source réelle, permettant des tests hors-ligne reproductibles.

Serverless.
Modèle d'exécution où le code s'exécute dans une fonction éphémère, sans serveur permanent. Le système de fichiers n'y persiste pas.

Étiquetage.
Mention visible signalant qu'un rendu est illustratif et non contractuel.

Porte de recherche (*grep gate*).
Vérification automatique qui échoue si un motif interdit (par exemple un champ de score) apparaît dans le code ou un schéma émis.

C.3 Sigles

Table C.1. Sigles et acronymes utilisés dans le rapport.

Sigle	Signification
API	Application Programming Interface
BAN	Base Adresse Nationale
BDNB	Base de Données Nationale des Bâtiments
BBOX	*Bounding box* — boîte englobante
CDS	Climate Data Store (Copernicus)
CORS	*Cross-Origin Resource Sharing*
CSTB	Centre Scientifique et Technique du Bâtiment
DPE	Diagnostic de Performance Énergétique
GML	Geography Markup Language
IGN	Institut Géographique National
INSEE	Institut National de la Statistique et des Études Économiques
LLM	*Large Language Model* — grand modèle de langage
OGC	Open Geospatial Consortium
PPR	Plan de Prévention des Risques
PPRI	Plan de Prévention du Risque Inondation
RCP	*Representative Concentration Pathway* — trajectoire de concentration représentative (scénario climatique)
REST	*Representational State Transfer*
RGA	Retrait-gonflement des argiles
SSP	Sites et sols pollués
SPA	*Single Page Application*
TMD	Transport de matières dangereuses
TRI	Territoire à Risque Important d'inondation
TTL	*Time To Live* — durée de vie (d'un cache)
VFX	*Visual effects* — effets visuels (mode de rendu cinématique)
WFS	Web Feature Service
WMS	Web Map Service

Sources et références

Sources de données utilisées

— **Géorisques** — portail national des risques naturels et technologiques, Ministère de la Transition écologique / BRGM. Données distribuées sous Licence Ouverte 2.0 (Etalab). https://www.georisques.gouv.fr

— **BDNB** — Base de Données Nationale des Bâtiments, Centre Scientifique et Technique du Bâtiment (CSTB). Licence Ouverte 2.0. https://api.bdnb.io

— **Géoplateforme** — service de géocodage, Institut Géographique National (IGN). https://data.geopf.fr/geocodage

— **Sandre** — portail national d'accès aux données sur l'eau. https://www.sandre.eaufrance.fr

— **Open-Meteo** — API météorologique ouverte, incluant le modèle hydrologique GloFAS. https://open-meteo.com

— **Panoramax** — service public de photographie de terrain contributive. https://panoramax.fr

— **Copernicus Climate Data Store** — projections climatiques européennes, ECMWF. Jeu de données *Climate indicators for Europe from 1940 to 2100*. https://cds.climate.copernicus.eu

— **Vigicrues** — service public de vigilance crues. https://www.vigicrues.gouv.fr

— **API Carto / BAN** — Base Adresse Nationale. https://adresse.data.gouv.fr

Cadres réglementaires

— **Licence Ouverte 2.0** (Etalab) — cadre juridique de réutilisation des données publiques françaises. Réutilisation commerciale autorisée, obligation d'attribution. https://www.etalab.gouv.fr/licence-ouverte-open-licence

— **Directive Inondation** — cadre européen ayant conduit à la définition des Territoires à Risque Important d'inondation (TRI).

— **Régime CatNat** — dispositif français d'indemnisation des catastrophes naturelles, article L125-1 du Code des assurances.

— **Solvabilité II** — directive européenne 2009/138/CE encadrant la solvabilité des organismes d'assurance et de réassurance.

Standards techniques

— **OGC Web Feature Service 2.0** — standard d'échange de données géographiques vectorielles.

— **OGC Geography Markup Language 3.2** — format d'encodage des géométries.

— **Mapbox GL JS** — bibliothèque de rendu cartographique.

— **FastAPI** — cadre de développement d'API Python.

— **Pydantic v2** — validation de données Python.

Documents internes au projet

— `constitution.md` — règles non négociables, version 0.2, amendée le 14 septembre 2026.

— `docs/workflow/01-brainstorm-notes.md` — journal des décisions de la phase 1.

— `docs/workflow/concept.md` — concept et questions ouvertes résolues (OQ-1 à OQ-6).

— `docs/workflow/spec.md` — spécification fonctionnelle, 28 exigences FR-01 à FR-28.

— `docs/workflow/plan.md` — plan d'exécution, tâches T001 à T018.

— `docs/workflow/scenario-plan.md` — plan du moteur de scénarios, tâches SCN-001 à SCN-050.

— `docs/workflow/vfx-spec.md` et `vfx-plan.md` — spécification et plan du mode de rendu cinématique.

— `AGENTS.md` — conventions, invariants acquis et pièges techniques.

— `handoff.md` — décisions architecturales en attente.

— `roadmapproduction.md` — feuille de route vers la production.

— `README.md` — documentation d'installation, d'usage et de déploiement.